

Relazione per
“Programmazione ad Oggetti”

Lorenzo Casadei
Martina Malagoli
Davide Mancini
Sara Visani

31 luglio 2025

Indice

1	Analisi	3
1.1	Descrizione e requisiti	3
1.2	Modello del Dominio	5
2	Design	7
2.1	Architettura	7
2.2	Design dettagliato	9
2.2.1	Davide Mancini	9
2.2.2	Martina Malagoli	14
2.2.3	Sara Visani	19
2.2.4	Lorenzo Casadei	29
3	Sviluppo	40
3.1	Test automatizzati	40
3.2	Note di sviluppo	41
3.2.1	Davide Mancini	41
3.2.2	Martina Malagoli	42
3.2.3	Sara Visani	42
3.2.4	Lorenzo Casadei	43
4	Commenti finali	44
4.1	Autovalutazione e lavori futuri	44
4.1.1	Davide Mancini	44
4.1.2	Martina Malagoli	45
4.1.3	Sara Visani	45
4.1.4	Lorenzo Casadei	46
4.2	Difficoltà incontrate e commenti per i docenti	46
4.2.1	Davide Mancini	46
4.2.2	Martina Malagoli	46

A Guida utente	47
A.1 Obiettivo del Gioco	47
B Esercitazioni di laboratorio	51
B.1 davide.mancini10@studio.unibo.it	51
B.2 martina.malagoli4@studio.unibo.it	51

Capitolo 1

Analisi

1.1 Descrizione e requisiti

L'applicazione consiste in un gioco platformer, ispirato a vari cult del genere come Castlevania, Kid Icarus, Super Mario e Spelunky. Un platformer è un gioco a scorrimento dove i personaggi si muovono su piattaforme. Nel caso dell'applicazione lo scorrimento sarà di tipo verticale. Lo scopo del gioco consiste nel raggiungere il fondo degli Inferi per recuperare Persefone dalla prigionia di Ade. Il giocatore può selezionare un personaggio fra cinque, ciascuno dei quali è distinto da abilità e caratteristiche speciali. Ogni personaggio si ispira ad una classe del gioco Dungeons&Dragons, ovvero: mago, ladro, druido e arciere.

La maggior parte delle abilità speciali dei personaggi richiede una certa quantità di mana per poter essere utilizzata. Con mana s'intende del potere spirituale che si consuma usando le abilità e che si recupera uccidendo i nemici.

Il personaggio può essere cambiato nel corso del gioco facendo ingresso all'interno di una particolare stanza del livello: le terme. All'interno delle terme, inoltre, è possibile salvare i progressi di gioco e recuperare punti vita e mana eventualmente persi precedentemente. Gli Inferi stessi sono considerati una stanza del gioco, in quanto parte dello stesso livello. Un ulteriore tipo di stanza è il negozio, dove il giocatore ha la possibilità di comprare degli oggetti che forniscono potenziamenti temporanei (pozioni) a consumo immediato come:

- Acquisizione punti vita temporanei;
- Acquisizione di mana temporaneo;
- Aumento della velocità di attacco;

- Aumento della velocità di movimento;
- Aumento della potenza di attacco.

All'interno del negozio saranno sempre presenti tre pozioni di tipologie diverse fra quelle disponibili, che si resettano uscendo e rientrando nella stanza.

La valuta tramite cui è possibile eseguire l'acquisto è lo stesso punteggio del giocatore, chiamato punti anima, che aumenta eliminando i nemici incontrati durante la partita. I nemici possono essere di quattro tipologie, due volanti e due terreni. Dei due nemici volanti il primo attacca il personaggio sparando fuoco che insegue il giocatore, l'altro invece attacca con proiettili con traiettoria fissa orientata verso il basso. I due nemici di terra, d'altra parte, attaccano entrambi con colpi fisici, ma se uno si muove lentamente l'altro rincorre il giocatore.

Una volta che un nemico viene sconfitto, oltre ad aumentare i punti del giocatore, esso ha una certa probabilità di lasciare un oggetto che può essere raccolto dal giocatore per guadagnare un potenziamento della stessa tipologia delle pozioni, sebbene di più breve durata.

Il terreno di gioco del livello può essere di tre tipologie. La prima, quella base, non condiziona in alcun modo il giocatore o i nemici, le altre due invece comportano delle difficoltà aggiuntive: la lava toglie vita, senza considerare eventuali punti vita temporanei, ogni volta che viene toccata, mentre le liane riducono la capacità di movimento.

Requisiti funzionali

- Il progresso del giocatore può essere salvato;
- Devono essere presenti diverse tipologie di nemici;
- Devono esserci più personaggi giocabili;
- Devono essere gestite le collisioni tra entità;
- Il movimento dei nemici deve essere autonomo;
- Corretta gestione del punteggio e del negozio per gli acquisti;
- Corretta gestione del recupero del mana e dei punti vita;
- I personaggi devono poter essere cambiati nel corso del gioco;

Requisiti non funzionali

- La finestra di gioco potrà essere ridimensionabile;
- La lettura dell'input da tastiera dovrà essere reattiva;
- L'interfaccia di gioco funzionerà allo stesso modo in diversi sistemi operativi.

1.2 Modello del Dominio

Il gioco prevede un livello all'interno del quale vi sono tre stanze, ovvero Hell, Springs e Shop. All'interno di esse sono contenute delle entità (game objects), le quali possono essere:

- Entità mobili: suddivise in giocatore (uno dei quattro personaggi giocabili), nemici (possono essere di diverse tipologie e in base ad esse varia il comportamento) e proiettili (possono variare in base all'entità sparante)
- Blocchi: ovvero le piattaforme su cui si possono muovere il personaggio e i nemici;
- Potenzianti: ottenuti dall'eliminazione dei nemici o acquistati nello shop.
- Oggetti interagibili: distinguibili in potions (acquistate nello shop), il character changer (un cristallo) e save point (una lapide);
- Mercante;
- Armi, che possono essere ravvicinate o a lungo raggio.

In particolare, Hell è caratterizzato dalla presenza di nemici e di bonus, Springs dal save point e dal character changer, e Shop da pozioni e mercante. I game objects devono poter collidere fra loro tramite l'utilizzo di collider. I progressi del giocatore vengono salvati in un game data dove verranno salvati il punteggio, il personaggio attuale e la sua posizione al momento del salvataggio.

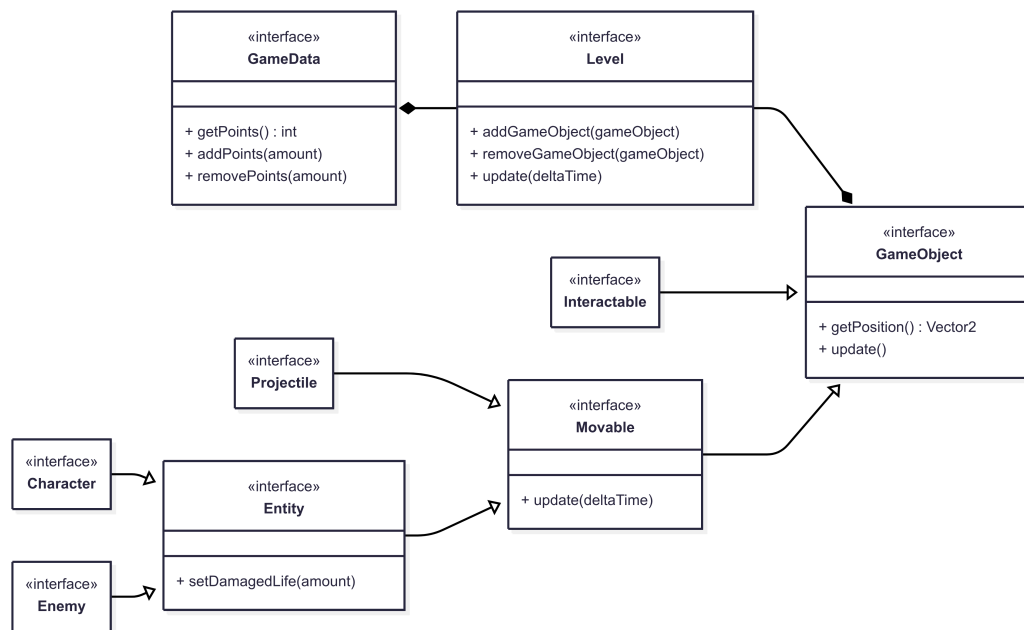


Figura 1.1: Schema UML dell'analisi del problema, con rappresentate le entità principali ed i rapporti fra loro

Capitolo 2

Design

2.1 Architettura

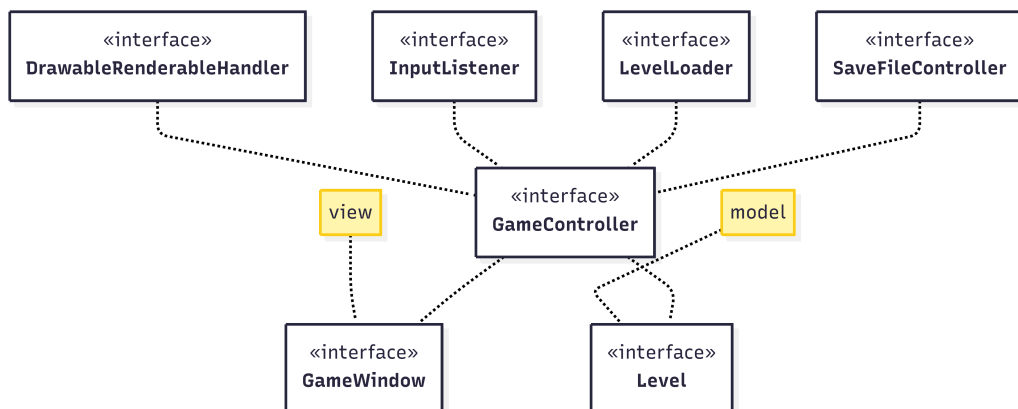


Figura 2.1: Schema UML che rappresenta l'architettura MVC del progetto

L'architettura dell'applicazione *Fall to Hell* segue il pattern architetturale *Model-View-Controller*. *Fall to Hell* implementa l'interfaccia **GameController** come controller del sistema e funge da cuore per la comunicazione tra il Model e la View.

La logica è data dall'interfaccia **Level**, che gestisce ogni funzione del modello di gioco. Il gioco contiene multipli game objects che posseggono informazioni inerenti a come dovrebbero essere mostrati a schermo e che, se necessario, possono richiedere al controller eventuali pulsanti della tastiera premuti tramite l'interfaccia **GameEventManager**. Quest'ultima astrae gli input da tastiera in eventi utilizzando l'interfaccia funzionale **GameEvent**. Per essere mostrati a schermo, i game objects possono eventualmente contenere

un'interfaccia **Drawable** che salva informazioni su come essere disegnati su schermo.

La lettura di input da tastiera, l'output mostrato a schermo e l'audio del gioco sono parte della View e sono assegnati all'interfaccia **GameWindow**. Per permettere al giocatore di sfruttare la tastiera per muovere il suo personaggio, il **GameController** legge l'input della tastiera dalla View tramite la classe **InputListener**, la quale potrà passare le informazioni appena ricevute al Model utilizzando l'interfaccia **GameEventManager**. L'audio del gioco è semplicemente una musica che inizia quando il giocatore avvia il gioco, la quale viene caricata dalla View e avviata nel Controller.

Ogni game object da mostrare a schermo deve avere un mezzo di comunicazione con la View, quindi l'applicazione utilizza l'interfaccia **RenderableController** per passare le informazioni di **Drawable** alla View, la quale utilizza l'interfaccia **Renderable** per mostrare a schermo le informazioni apprese da **Drawable**. Tutti i **RenderableController** sono salvati e gestiti da **DrawableRenderableHandler** che controlla la comunicazione tra **Drawable** e **Renderable**.

Il modello richiede anche accesso a file di testo, contenenti informazioni per salvare il gioco e risorse come il livello e gli oggetti del negozio. Per questo motivo l'interfaccia **FileController** viene utilizzata per interfacciarsi con il sistema operativo e legge un file in un certo percorso relativo. Per salvare il gioco viene utilizzata l'interfaccia **SaveFileController** per salvare e caricare un file che mantiene le informazioni della sessione di gioco corrente che viene salvato all'interno della cartella utente del giocatore. Per leggere un'immagine da file si utilizza l'interfaccia **ImageController** che restituisce un'immagine che verrà successivamente utilizzata dalla View, mentre per leggere un file audio si utilizza la classe **SoundPlayerView**. L'interfaccia **AudioManager** mantiene le informazioni sulla musica per poterla avviare e fermare dal Controller. L'applicazione offre anche la possibilità di mettere in pausa il gioco.

2.2 Design dettagliato

2.2.1 Davide Mancini

Costruire un livello contenente informazioni facoltative

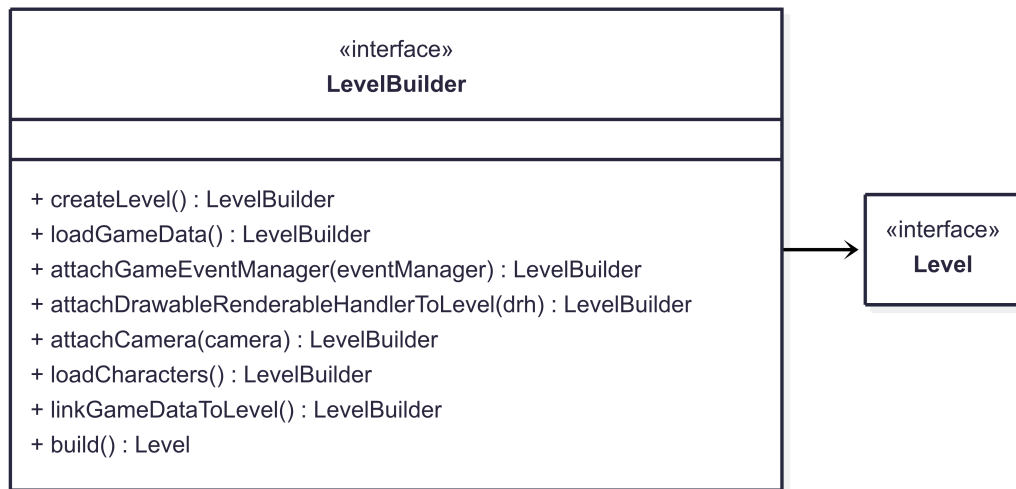


Figura 2.2: Schema UML rappresentante l'uso di Builder per costruire un livello con parametri di base oppure scelti in base alle richieste di utilizzo

Problema Si deve poter costruire un livello senza tutte le sue componenti per funzionare.

Soluzione La soluzione propone di utilizzare il *pattern Builder*, come da Figura 2.2, per costruire un **Level** con più componenti opzionali, i quali, se non inseriti, verranno sostituiti con parametri di base.

Multiple classi dei personaggi

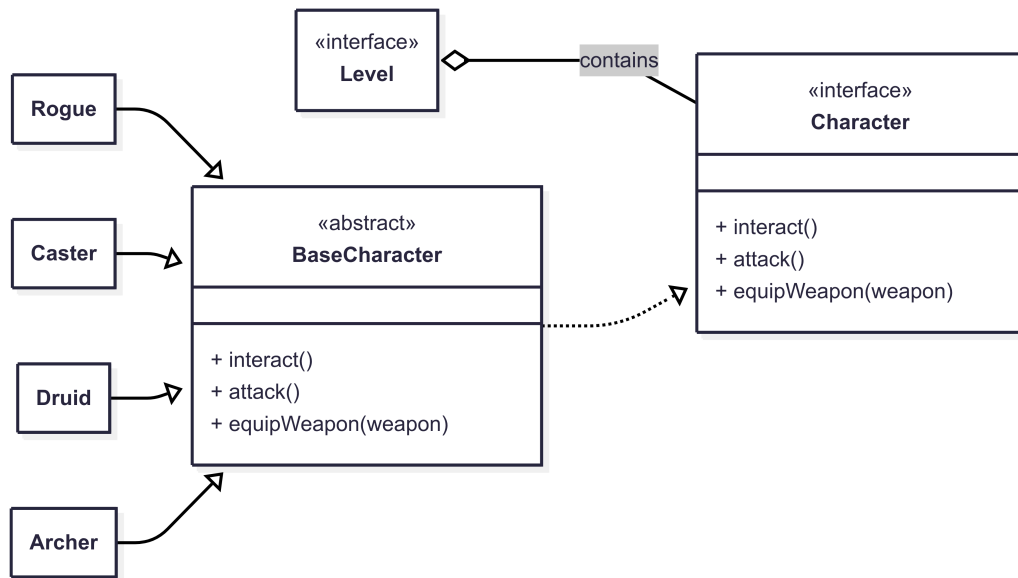
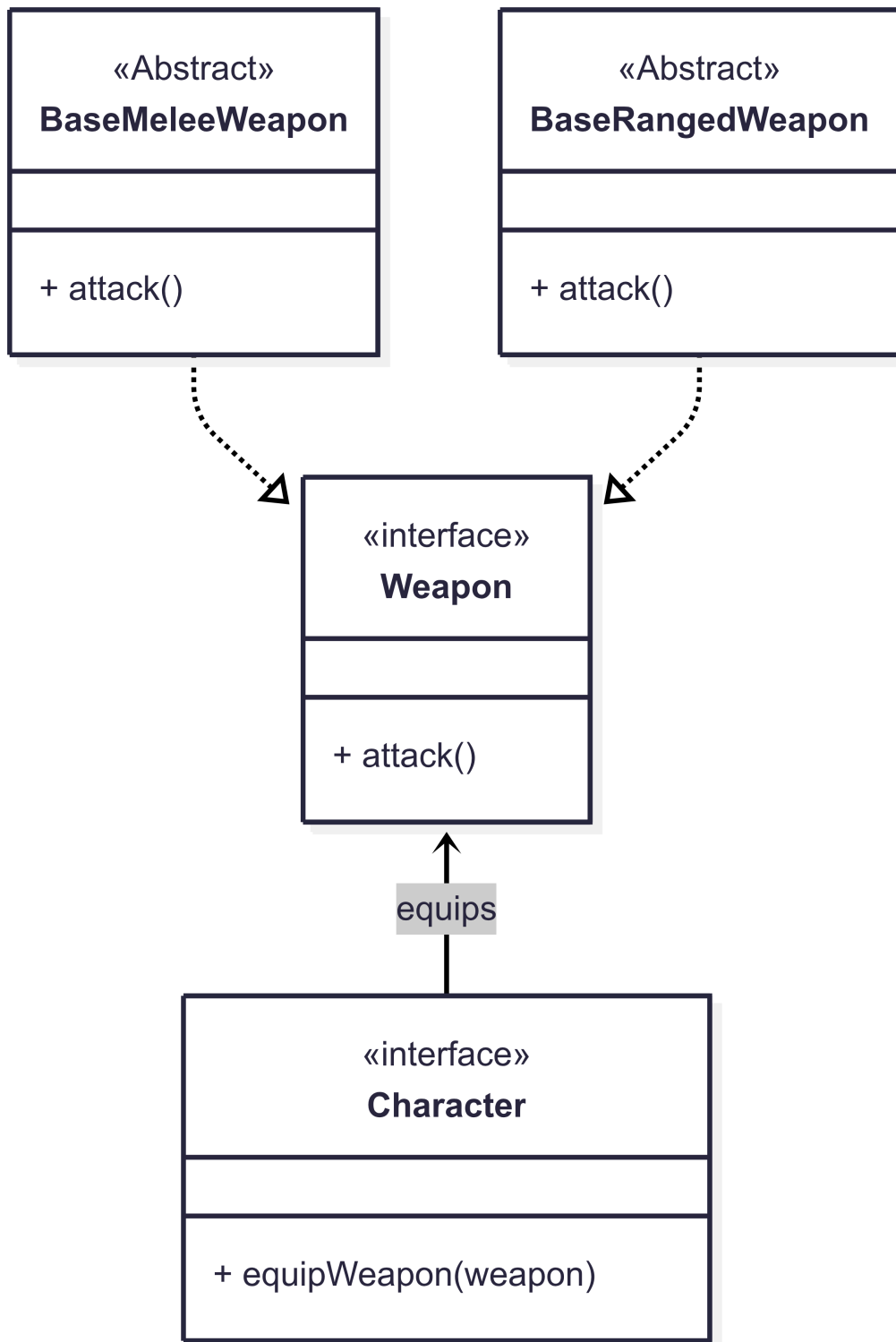


Figura 2.3: Schema UML rappresentante l'uso di Builder per costruire un livello con parametri di base oppure scelti in base alle richieste di utilizzo

Problema Il giocatore deve poter utilizzare personaggi di diverse classi, con armi e abilità diverse.

Soluzione Nell'implementazione si è scelto di applicare il *pattern Strategy*, come da Figura 2.3, dove la strategia è **Character** e la classe che utilizza la strategia è il livello che lo contiene. Per evitare ripetizioni di codice per funzionalità comuni, si è utilizzata una classe astratta **BaseCharacter** per gestire l'equipaggiamento dell'arma, il suo attacco e il movimento (in base ai comandi del giocatore), che sono comuni ad ogni personaggio.

Equipaggiamento di armi da parte dei personaggi



Problema Il personaggio del giocatore appartiene ad una classe. La classe determina l'arma posseduta, che può essere di più tipologie.

Soluzione La soluzione adottata è quella di utilizzare il *pattern Strategy*, come da Figura 2.4. La strategia è **Weapon**, che può essere **RangedWeapon** o **MeleeWeapon**, mentre la classe che utilizza la strategia è **Character**. Questa scelta permette di aggiungere in futuro nuove tipologie di armi e classi.

Collisioni tra GameObjects

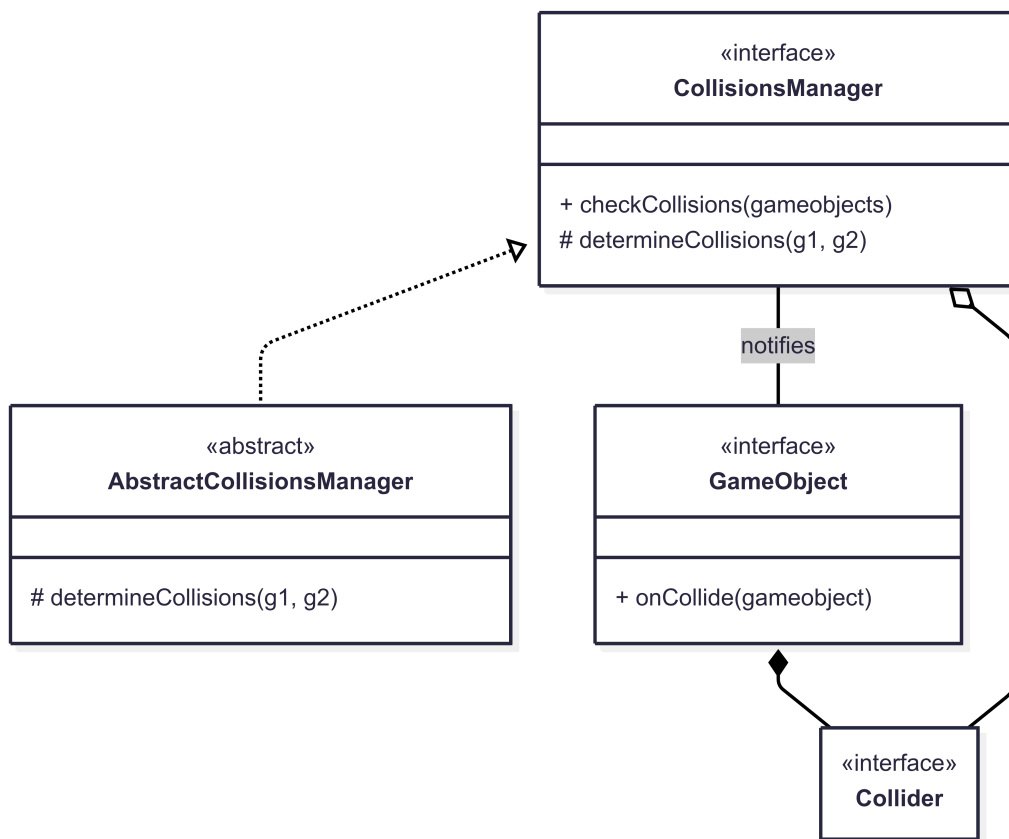


Figura 2.5: Schema UML rappresentante l'uso di Observer per notificare la collisione di due oggetti

Problema Ogni **GameObject** può andare in contatto con altri **GameObject** in un qualsiasi momento del gioco. Deve essere possibile gestire ogni collisione tra game objects individualmente nella loro implementazione.

Soluzione Per permettere di lasciare al `CollisionsManager` il compito di controllare quando dei game objects collidono e di avvisarli della collisione, la soluzione proposta utilizza il *pattern Observer*, come da Figura 2.5. L'observer è `GameObject` e il subject è `CollisionsManager`, il quale avviserà quando due `GameObject` andranno in contatto tra loro. Sarà il `GameObject` stesso a decidere cosa fare, una volta notificato, all'interno del metodo `onCollision()`. Inoltre, il `CollisionsManager`, per avere una maggiore flessibilità, viene implementato in `AbstractCollisionsManager` utilizzando il *pattern Strategy*. La strategia è *CollisionsManager*, mentre la classe che la utilizza è `AbstractCollisionsManager`.

Algoritmi diversificati in base alla tipologia di collider

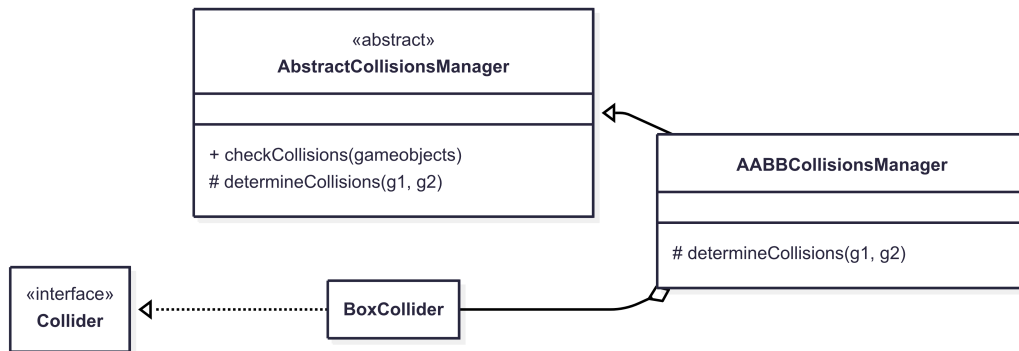


Figura 2.6: Schema UML rappresentante l'uso di Template per utilizzare un algoritmo in base alla tipologia dei collider

Problema I `Collider` devono essere di più tipologie, quindi devono essere possibili più implementazioni dell'algoritmo per controllare le collisioni.

Soluzione Poiché i `Collider` possono essere di più tipologie, la classe astratta `AbstractCollisionsManager` può avere più implementazioni seguendo il *pattern Template method*, come da Figura 2.6. Il metodo template è `checkCollisions()`, che chiama un metodo astratto e protetto `determineCollision()`.

2.2.2 Martina Malagoli

Gestione dei timer

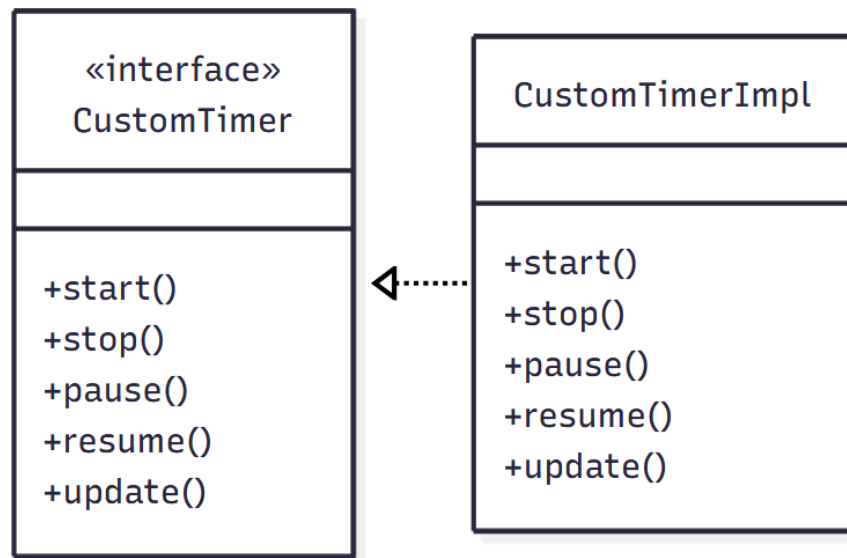


Figura 2.7: Schema UML rappresentante l'uso di Strategy per la realizzazione di CustomTimer

Problema Vari eventi all'interno dell'applicazione sono legati al tempo e dipendono da specifiche scadenze. Deve quindi essere possibile avere una gestione dello scorrere del tempo.

Soluzione La soluzione proposta è quella dell'utilizzo di timer, `CustomTimer`. Essa consiste nell'utilizzo del *pattern Strategy*. Infatti, sebbene sia stato implementato una sola tipologia di `CustomTimer`, tale scelta permette di avere più flessibilità e di poter agilmente realizzare ed utilizzare diverse tipologie di timer che utilizzano diversi algoritmi, rendendoli potenzialmente interscambiabili per le classi che ne fanno utilizzo. In questo caso la strategia utilizzata è quindi `CustomTimer` e l'utilizzatore della strategia è chiunque decida di utilizzare `CustomTimer`.

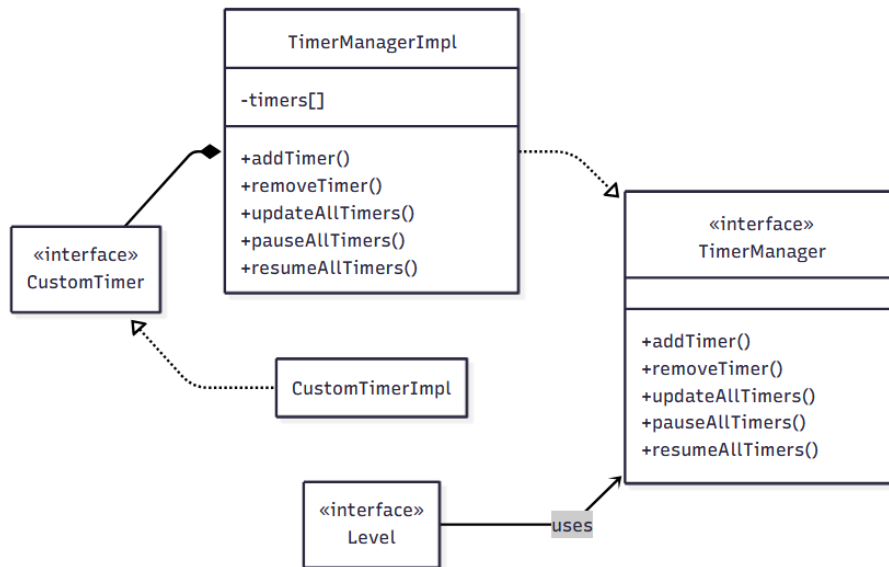


Figura 2.8: Schema UML rappresentante l'uso di Strategy e Observer per la realizzazione di `TimerManager`

Problema Il gioco per poter funzionare deve poter utilizzare timer multipli per gestire scadenze ed eventi dipendenti dal tempo. Dunque deve essere possibile gestire sia un unico timer che il loro insieme.

Soluzione Per facilitare l'utilizzo dei timer e la loro gestione è stato utilizzato il `TimerManager`. Esso ha il compito di salvare tutti i timer utilizzati per poterli inizializzare, bloccare, mettere in pausa, riavviarli e rimuoverli. La soluzione proposta utilizza in parte il *pattern Observer*: `TimerManager` è il subject, mentre i vari `CustomTimer` sono gli observer. `TimerManager`, infatti, notifica i vari `CustomTimer`, riuscendoli a gestire contemporaneamente, facendo il loro update ed eventualmente mettendoli tutti in pausa o riavviandoli. D'altra parte è stata però resa possibile anche la gestione singola dei timer. `TimerManager`, inoltre, sfrutta anche il *pattern Strategy*, in quanto è possibile implementare diversi tipi di timer manager che possono essere utilizzati dal livello `Level`. In questo caso `TimerManager` è la strategia e chi usa la strategia è `Level`.

Item del negozio

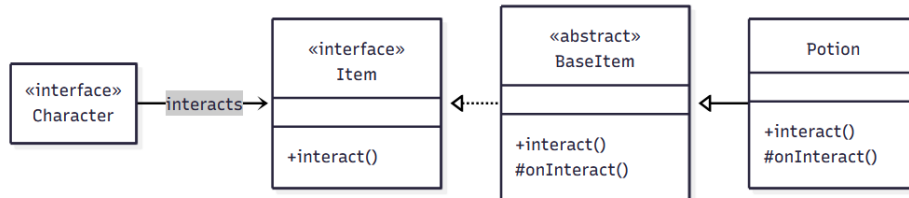


Figura 2.9: Schema UML rappresentante l'uso di Strategy e Template method per la gestione degli oggetti del negozio

Problema All'interno del negozio dell'applicazione possono essere presenti diverse tipologie di oggetti acquistabili, con caratteristiche fra loro differenti ma con vari elementi comuni, come il fatto di poter essere o meno acquistabili e il fatto di essere interagibili.

Soluzione Per risolvere il problema della creazione di diverse tipologie di oggetti del negozio si è proposta come soluzione l'utilizzo del *pattern Template method*. La classe **BaseItem** è la classe astratta, il cui metodo astratto `onInteract` può essere sovrascritto dalle sue sottoclassi, come per esempio **Potion**. Questa scelta permette di aggiungere nuovi item con diverse caratteristiche sfruttando la struttura di **BaseItem**. Qui il metodo template è `interact()`, quale chiama il metodo `onInteract()`.

Problema Un personaggio durante il gioco può potenzialmente interagire con diverse tipologie di oggetti del negozio.

Soluzione Per risolvere questo problema è possibile utilizzare il *pattern Strategy*, dove **Item** è la strategia e **Character** è chi utilizza la strategia. In questo modo si ha che un **Character** può interagire con vari **Item** in modo flessibile, permettendo in futuro anche l'aggiunta di nuovi **Item**.

Gestione dei potenziamenti

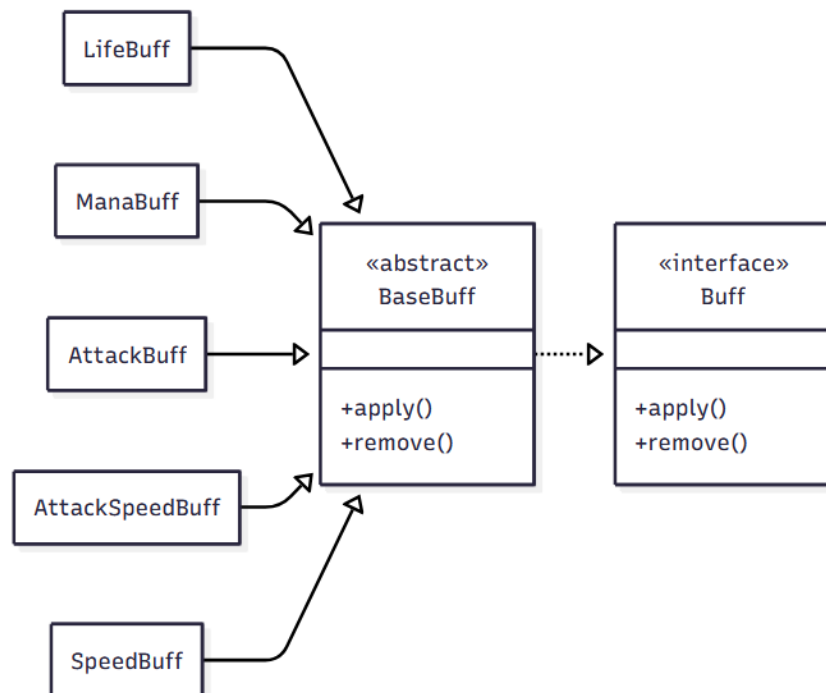


Figura 2.10: Schema UML rappresentante l'uso di Strategy per la realizzazione di diversi tipi di Buff

Problema I potenziamenti all'interno del gioco possono essere di varia natura e presentano caratteristiche peculiari.

Soluzione Per risolvere i problemi causati dal fatto di avere differenti tipologie di Buff si è scelto di utilizzare il *pattern Strategy*. In questo caso la strategia è Buff e l'utilizzatore della strategia diventa chiunque decida di utilizzare Buff, come per esempio *Potion* o *BuffManager*. Ciò permette una maggior estendibilità, infatti sarà possibile decidere di utilizzare diverse e nuove tipologie di Buff senza dover modificare la struttura delle classi che sfruttano tale strategia.

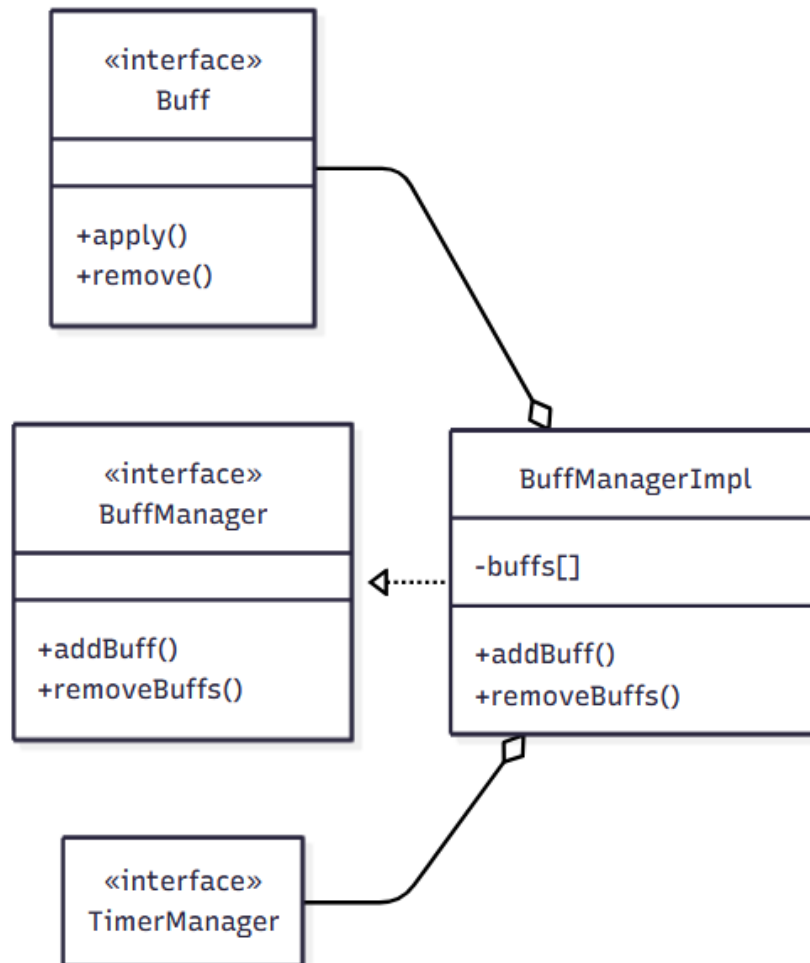


Figura 2.11: Schema UML rappresentante la gestione dei Buff tramite il BuffManager

Problema Durante il gioco possono venir applicati vari potenziamenti al personaggio corrente i quali devono essere gestiti nella loro aggiunta e successiva rimozione.

Soluzione Per poter gestire i vari Buff viene utilizzato un **BuffManager**. Tale soluzione non rispetta un particolare pattern specifico, ma mantiene al suo interno l'insieme dei Buff gestendone l'attivazione e la rimozione. Per quanto riguarda l'attivazione di un Buff, il **BuffManager** oltre che salvare tale

elemento si occupa eventualmente di notificare il `TimerManager` di avviarne un timer per la scadenza. Per quanto riguarda la rimozione essa può essere gestita dal `BuffManager` sia a livello dei singoli `Buff` nel caso in cui essi non siano vincolati da un timer, che del loro insieme, è infatti resa possibile la rimozione di tutti i `Buff` contemporaneamente.

2.2.3 Sara Visani

Creazione e gestione dei nemici

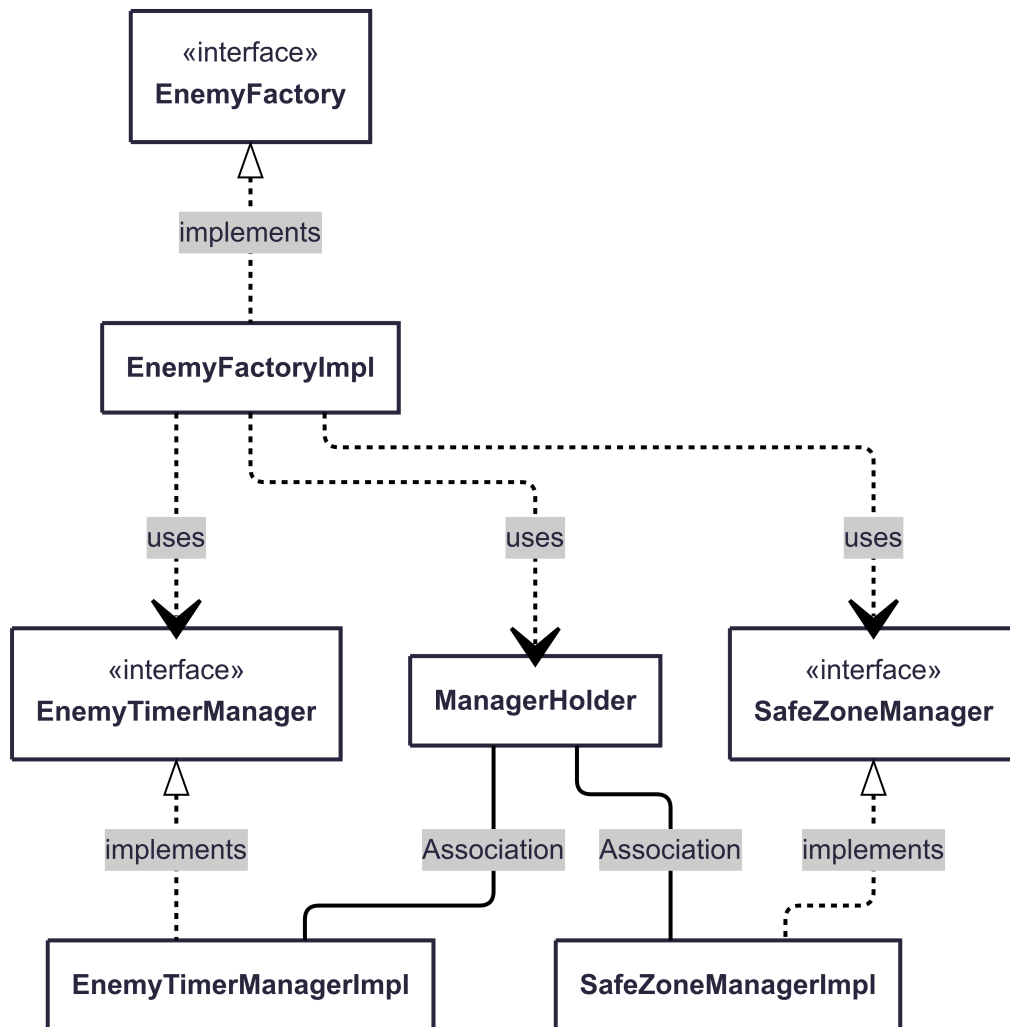


Figura 2.12: UML che mostra l'Abstract Factory Pattern

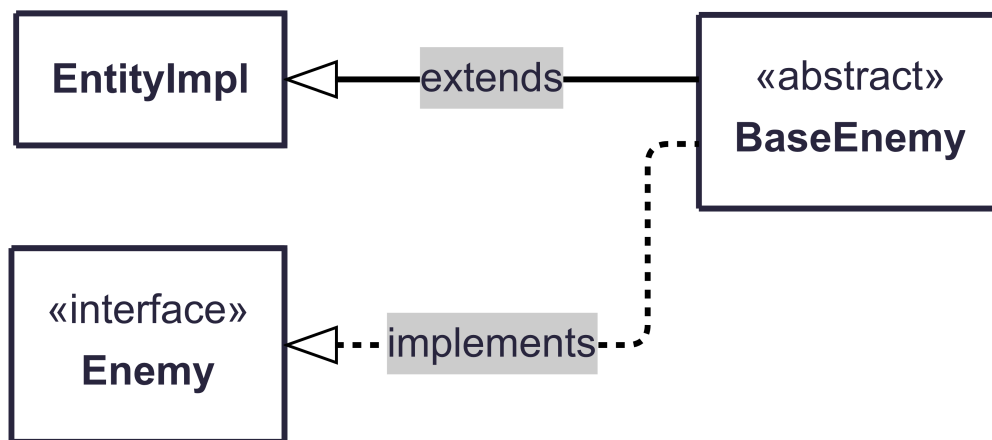


Figura 2.13: UML che mostra come BaseEnemy è collegato

Problema Creazione di vari tipi di nemici.

Soluzione Tutti i nemici sono sotto l'interfaccia **Enemy** e per evitare ripetizione di codice dei metodi comuni c'è una classe astratta **BaseEnemy** la quale può essere implementata come uno desidera. In questo caso sono stati implementati quattro tipi di nemici, i quali vengono creati mediante una abstract factory. Questa classe seguendo il pattern *Abstract Factory* ha all'interno una cache, lazy-loaded singleton per livello, per dare ai nemici due manager i cui sono comuni a tutti i nemici di un livello.

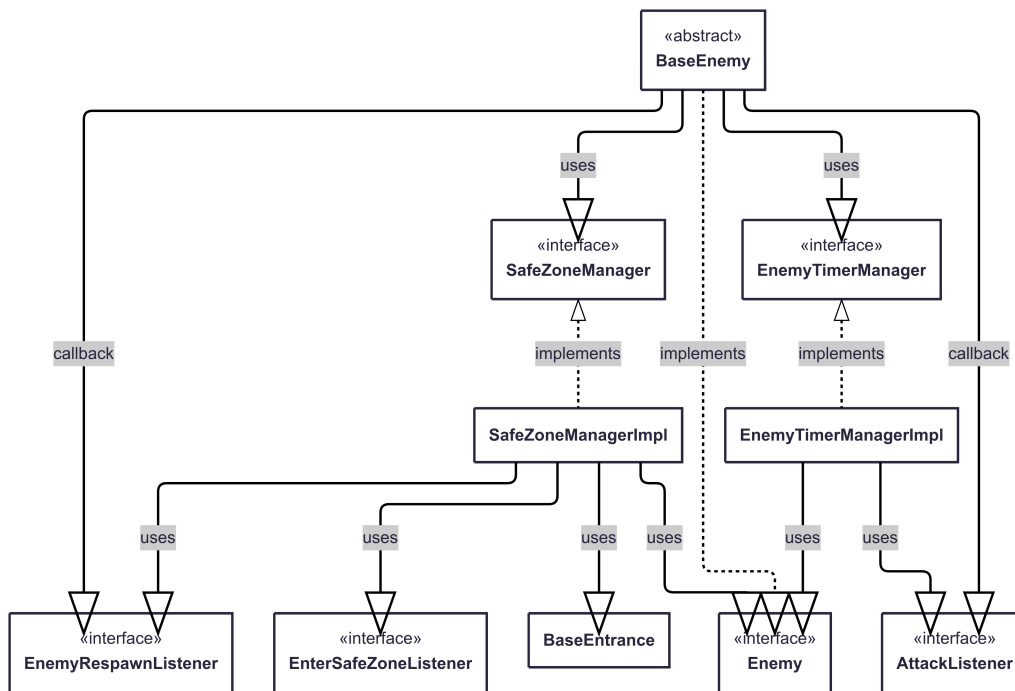


Figura 2.14: Questa immagine mette in evidenza il pattern listener tra gli Enemy e i managers

Problema Maneggiare le identificazioni e la vita dei timer di N nemici qualsiasi e farli scomparire e ricomparire quando il giocatore entra in una zona sicura.

Soluzione Per questo problema sono presenti due manager per livello. **EnemyTimerManager** prende il compito di creare, eliminare, startare e cercare un qualsiasi timer inerente ai nemici. Invece **SafeZoneManager** ha il compito di avvertire i nemici quando il giocatore entra o esce da una area sicura. Per far questo entrambi hanno dei listener o lambde da richiamare. Questa soluzione usa il *Pattern Listener*.

Creazione e Gestione dei Famigli del Druido

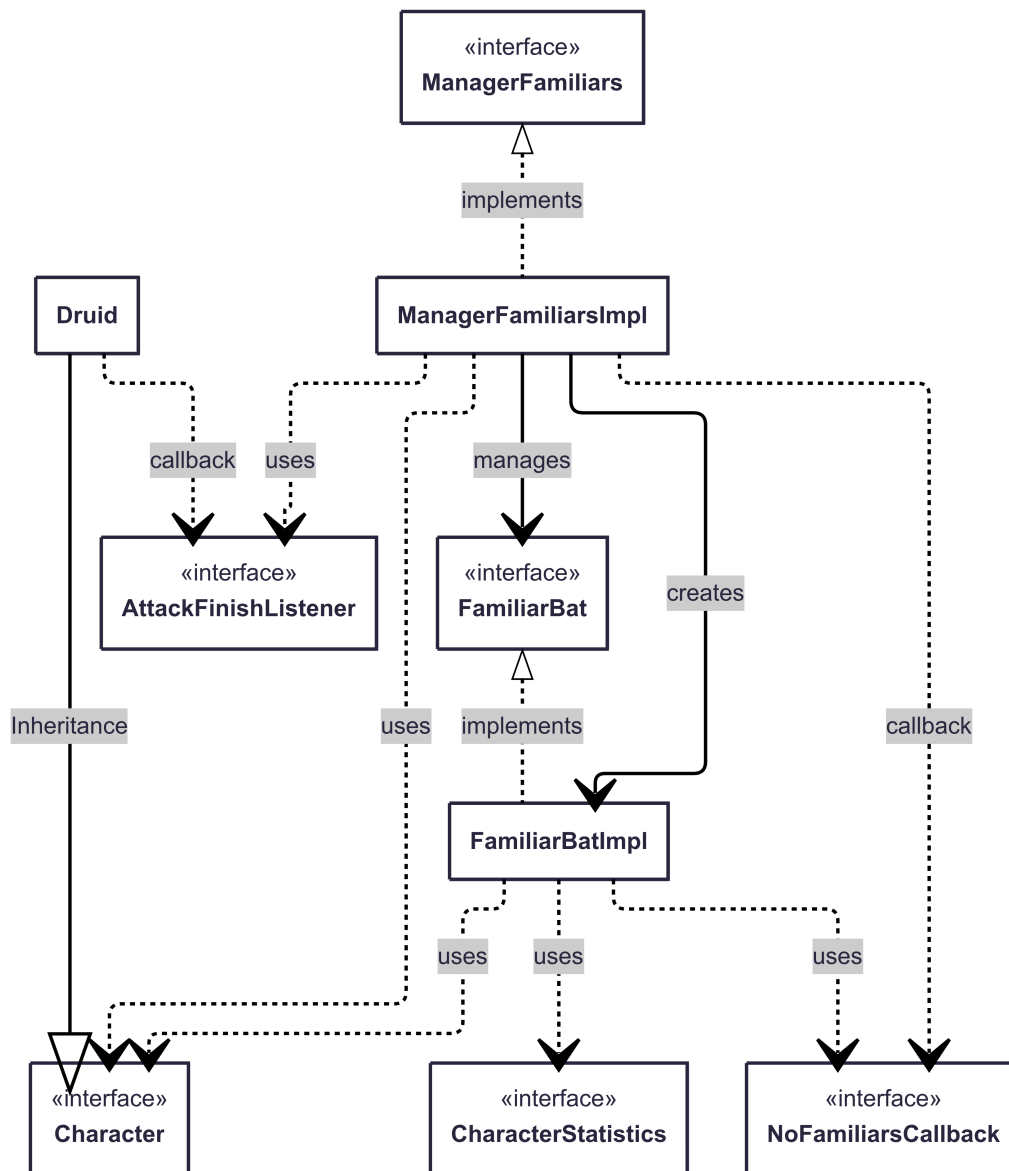


Figura 2.15: La figura mette in evidenza le relazioni tra le classi nominate a fianco con il pattern Listener

Problema Creazione e gestione dei famigli (attacco, movimento e vita) inoltre avvertire il druido quando può ordinare l'attacco e quando non ha più famigli a disposizione.

Soluzione Anche in questa soluzione è stato introdotto un manager con due listener. Uno per avvertire il `Druid` che non ha più famigli e l'altro chiamato da `FamiliarBat`, il quale informa al `ManagerFamiliars` che ha finito di attaccare. Questo serve perché i famigli hanno una vita dettata da timer ma se stanno attaccando prima finiscono l'attacco e poi scompaiono. Inoltre il motivo per il `Druid` va avvertito se non ha famigli è per non sprecare mana per un attacco che non si può attivare.

Creazione delle Abilità

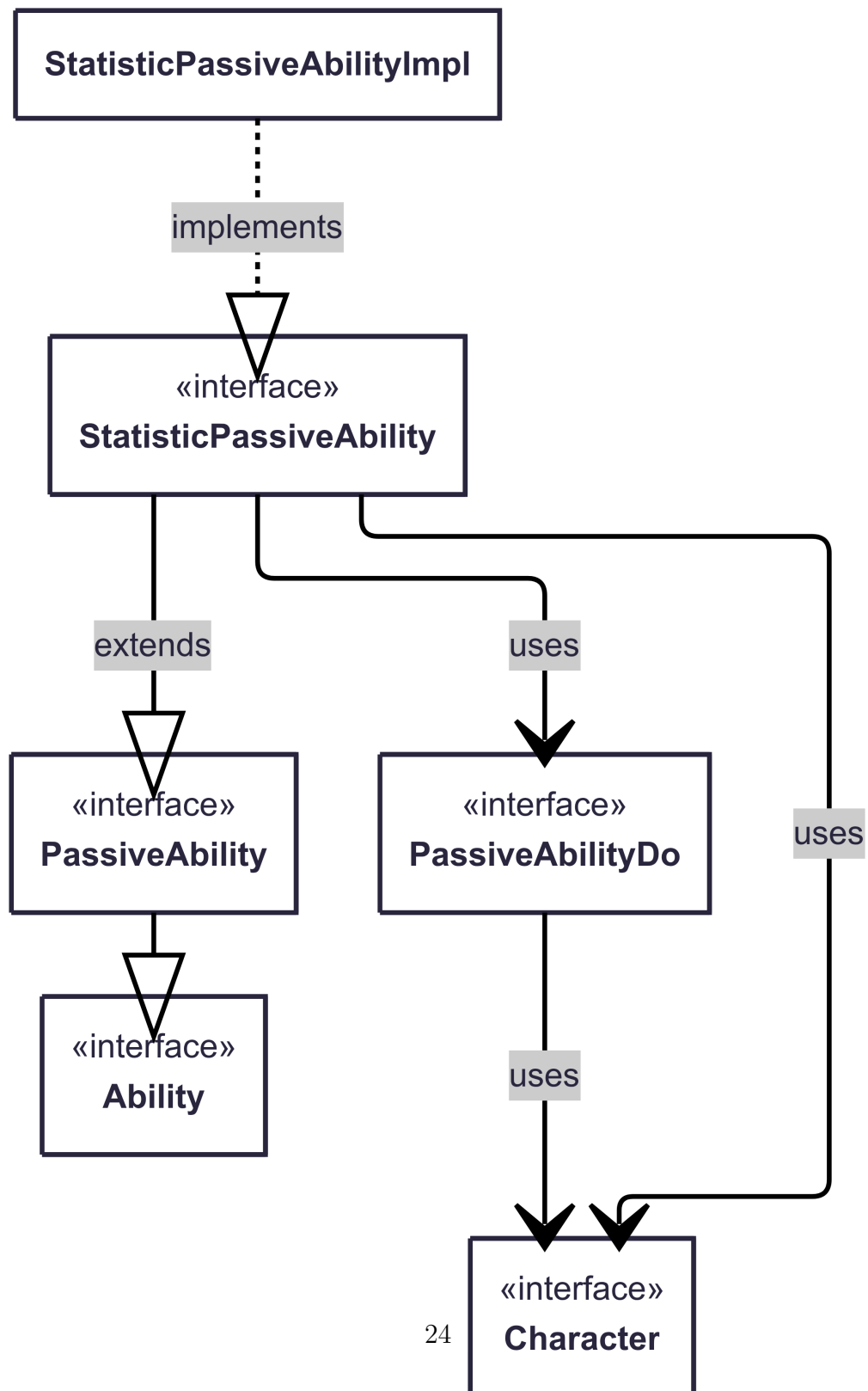


Figura 2.16: La figura mette in evidenza il Pattern Strategy nel ramo delle abilità passive

Problema Creazione di abilità con comportamenti diversi.

Soluzione Per risolvere questo problema ho sfruttato il *pattern Strategy*. Il quale è perfetto per questo caso proprio perché è solo l'azione che la `PassiveAbility` che cambia. `PassiveAbilityDo` è l'interfaccia `Strategy` le cui varie implementazioni si trovano dentro i vari personaggi, io ho implementato la strategy solo del druido.

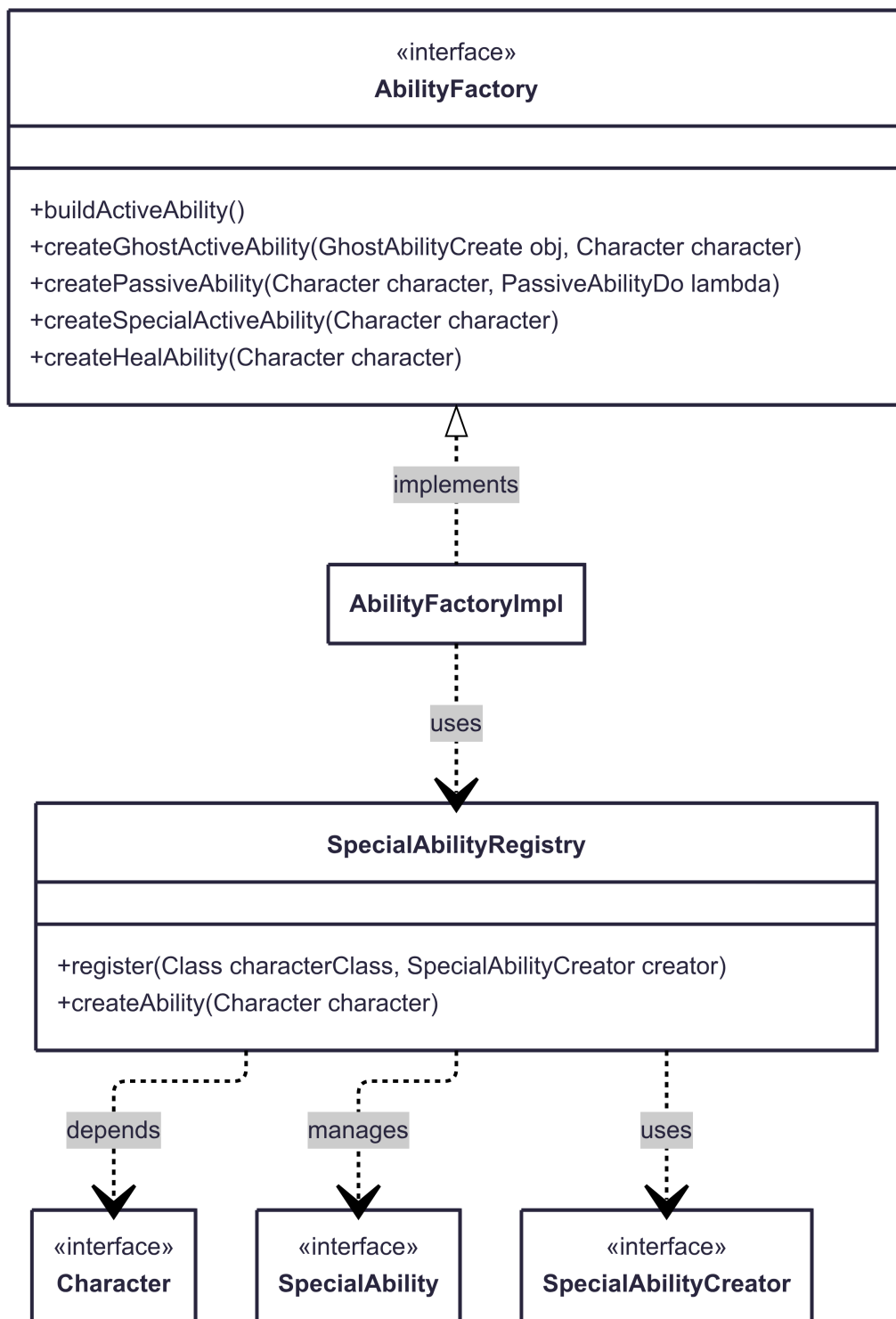


Figura 2.17: Rappresenta la Factory + Strategy usate per le abilità

Problema Creazione di diversi tipi di abilità anche di classi specifiche per sottoclassi di personaggio.

Soluzione Uso del *pattern Factory* con una modifica, all'interno è presente un **Register** che si occupa a dare alla factory l'abilità giusta se la factory non la sa, quindi è il pattern *Factory + Strategy*. Questa classe register serve per salvare al suo interno una mappa delle classi con il corrispondente oggetto richiesto. Quando la factory lo chiama gli dice la classe del **Character** che ha e viene restituito l'abilità giusta (questo solo per la **SpecialAbility**).

Creazione e Gestione delle Statistiche delle Entità

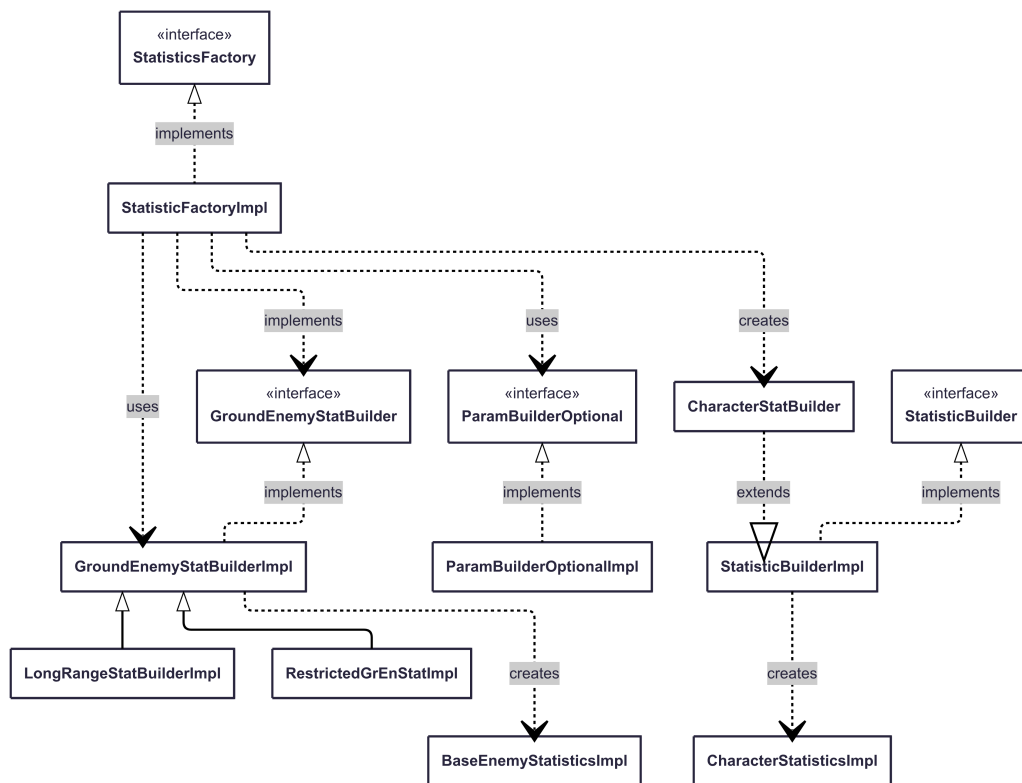


Figura 2.18: Rappresenta il Pattern ibrido Factory + Builder

Problema Creazione delle statistiche.

Soluzione La gestione delle statistiche è stata portata fuori dalle entità in una serie di oggetti gerarchici che costruiscono i necessari tipi di statistiche

per le varie sottoclassi di **Entity**. Oltre a questo le alcune statistiche possono contenere dei campi opzionali per questo motivo serve un costruttore che controlla che vengono messi tutti i campi obbligatori, ma che sia abbastanza fluido per i campi non obbligatori. Per questo problema ho deciso di usare un pattern ibrido *Factory + Builder*, quest'ultimo è in forma gerarchica e fluida *Fluent Builder* usando CRTP.

2.2.4 Lorenzo Casadei

Movable e GameObject

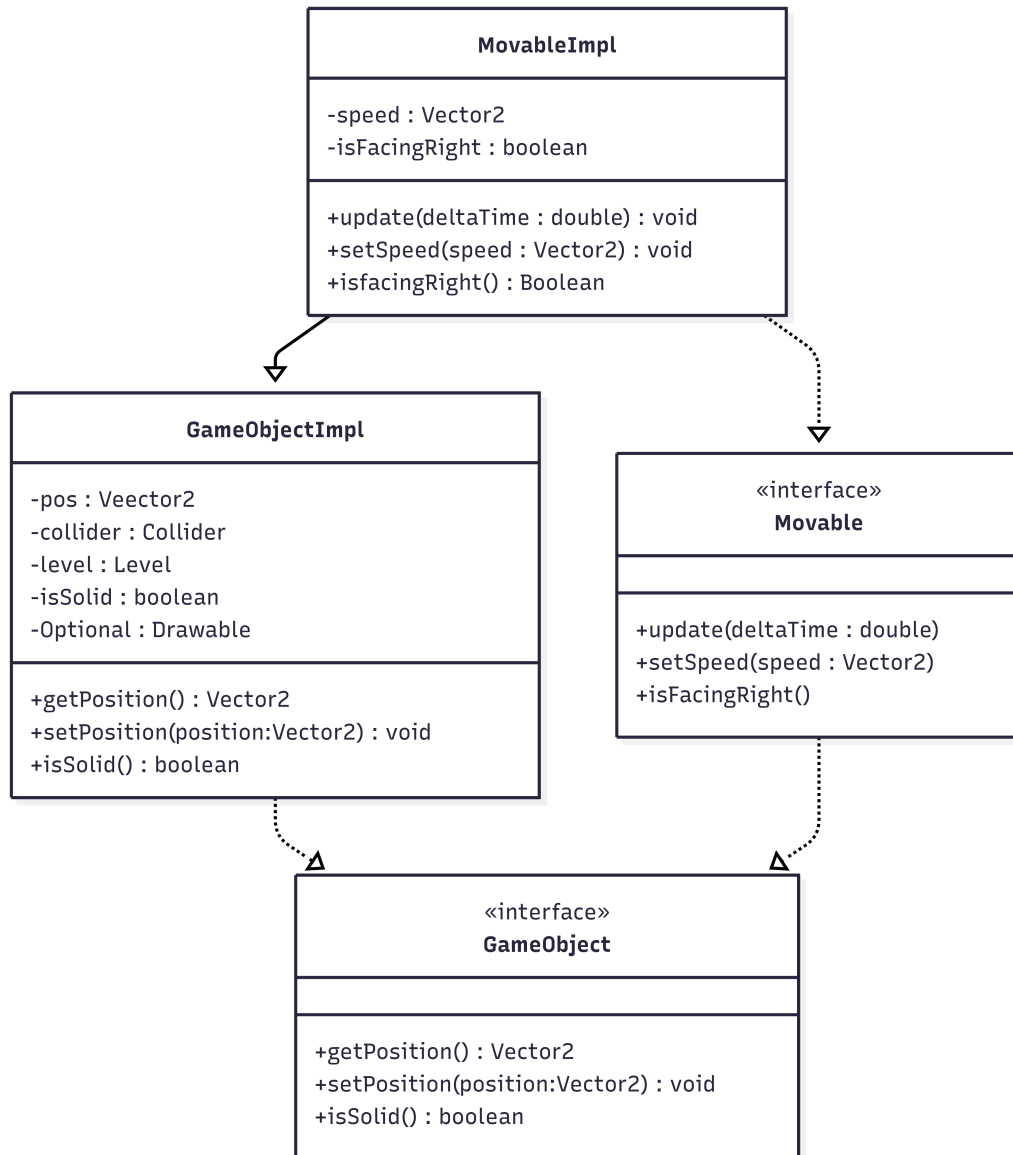


Figura 2.19: Schema UML rappresentante GameObject e Movable

Problema Nel contesto di un platform 2D, alcuni oggetti devono essere in grado di muoversi nel mondo di gioco con una velocità variabile. Inoltre, ogni oggetto deve conoscere la propria posizione e poter rilevare eventuali

collisioni. Era necessario creare una gerarchia di oggetti che permettesse di distinguere tra oggetti statici e oggetti mobili, riutilizzando il più possibile la logica comune (es. gestione posizione, livello, disegni).

Soluzione La soluzione adottata è stata creare una classe `GameObjectImpl`, che rappresenta un generico oggetto di gioco e implementa l'interfaccia `GameObject`. Essa incapsula proprietà comuni a tutti gli oggetti del mondo (posizione, collider, livello, solidità, ecc.) e gestisce automaticamente l'aggiunta al livello. Per gli oggetti mobili, è stata creata una sottoclasse `MovableImpl`, che estende `GameObjectImpl` e implementa l'interfaccia `Movable`. Tale classe aggiunge un vettore velocità e sovrascrive il metodo `update()` che aggiorna la posizione in base alla velocità e al tempo trascorso. Questa struttura permette un'estensione modulare: ad esempio, `Player`, `Enemy` o altri tipi di entità mobili possono derivare da `MovableImpl` e personalizzarne il comportamento. Non è stato applicato un design pattern formale per questa parte. Tuttavia, la struttura segue i principi di programmazione orientata agli oggetti.

Gestione del mondo di gioco

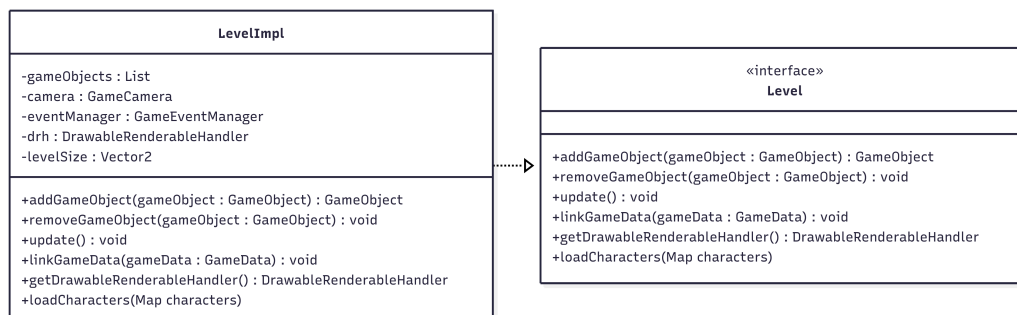


Figura 2.20: Schema UML rappresentante Level e LevelImpl

Problema Era necessario creare un contenitore per tutti gli oggetti di gioco, che fosse responsabile dell'aggiornamento degli oggetti mobili, del controllo delle collisioni e dell'interazione con altri componenti come la telecamera, gli eventi e l'interfaccia grafica. In particolare:

- Gli oggetti devono potersi registrare nel livello.
- La camera deve seguire il giocatore.
- Devono avvenire collisioni tra gli oggetti.
- Le entità devono poter ricevere eventi di gioco.

Soluzione È stata sviluppata la classe `LevelImpl`, che implementa l'interfaccia `Level`. Essa mantiene una lista degli oggetti di gioco, gestisce le collisioni usando un `CollisionsManager`, aggiorna lo stato degli oggetti tramite il metodo `update()` e integra il sistema eventi (`GameEventManager`) e la gestione dei disegni (`DrawableRenderableHandler`). Il livello funge da “centro di controllo” del gioco, delegando ad altri componenti funzionalità specifiche.

GameCamera

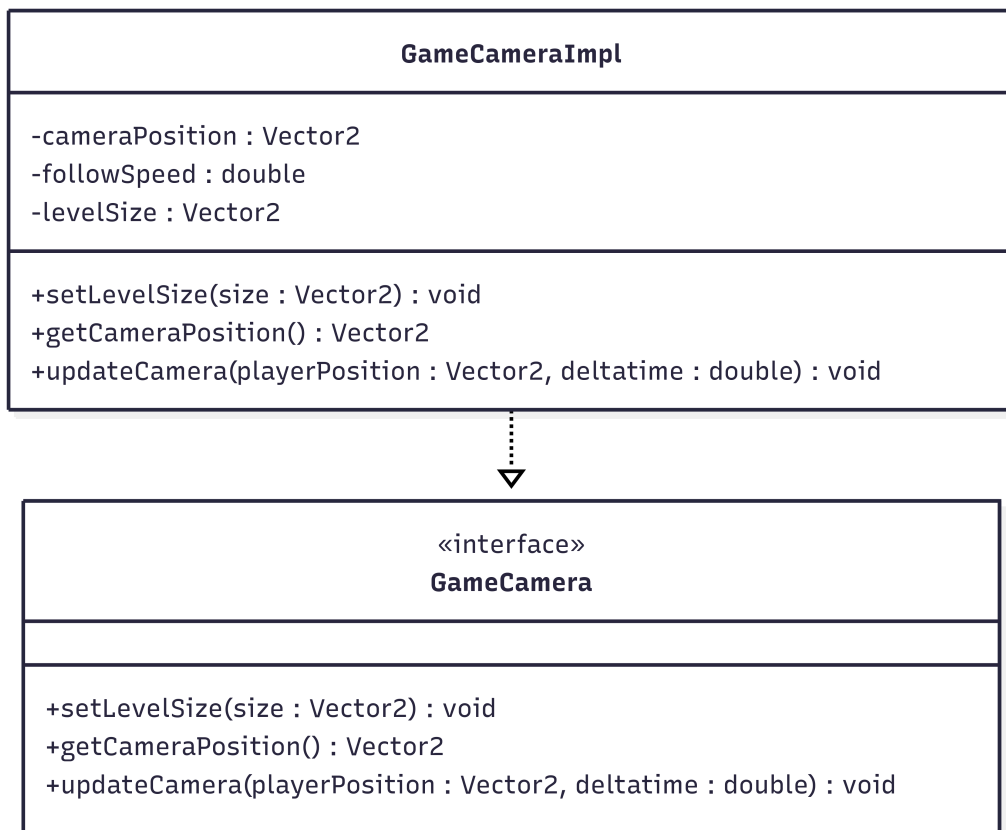


Figura 2.21: Schema UML rappresentante GameCamera

Problema In un platform 2D, è necessario che la “camera” segua il giocatore, mantenendolo al centro dello schermo quando possibile, ma senza uscire dai limiti del livello. Il movimento della camera deve essere fluido e adattabile.

Soluzione È stata implementata la classe `GameCameraImpl`, che tiene traccia della posizione, dimensioni visibili e velocità di inseguimento. Il metodo `updateCamera()` sposta la camera gradualmente verso la posizione del giocatore, limitandone lo spostamento ai bordi del livello. Questa separazione consente di usare la camera in modo riusabile e testabile, svincolandola dalla logica del personaggio o del livello.

GameEventManager

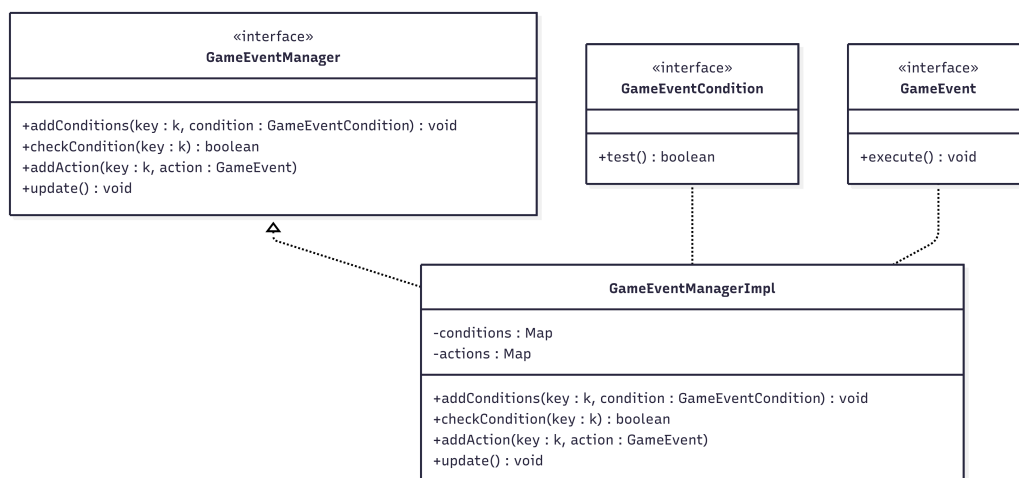


Figura 2.22: Schema UML rappresentante GameEvent

Problema Il gioco richiede la possibilità di reagire a condizioni che si verificano durante esso, come "è stato raccolto un oggetto". Era necessario un sistema generico che permettesse di associare a ogni condizione un'azione da eseguire.

Soluzione È stato progettato `GameEventManager<K>`, una classe parametrica che utilizza due mappe:

- Una per le condizioni (`GameEventCondition`, con metodo `test()`).
- Una per le azioni (`GameEvent`, con metodo `execute()`).

Nel metodo `update()` vengono iterate tutte le condizioni e, se vere, viene eseguita l'azione corrispondente.

Sistema di armi a distanza e gestione proiettili

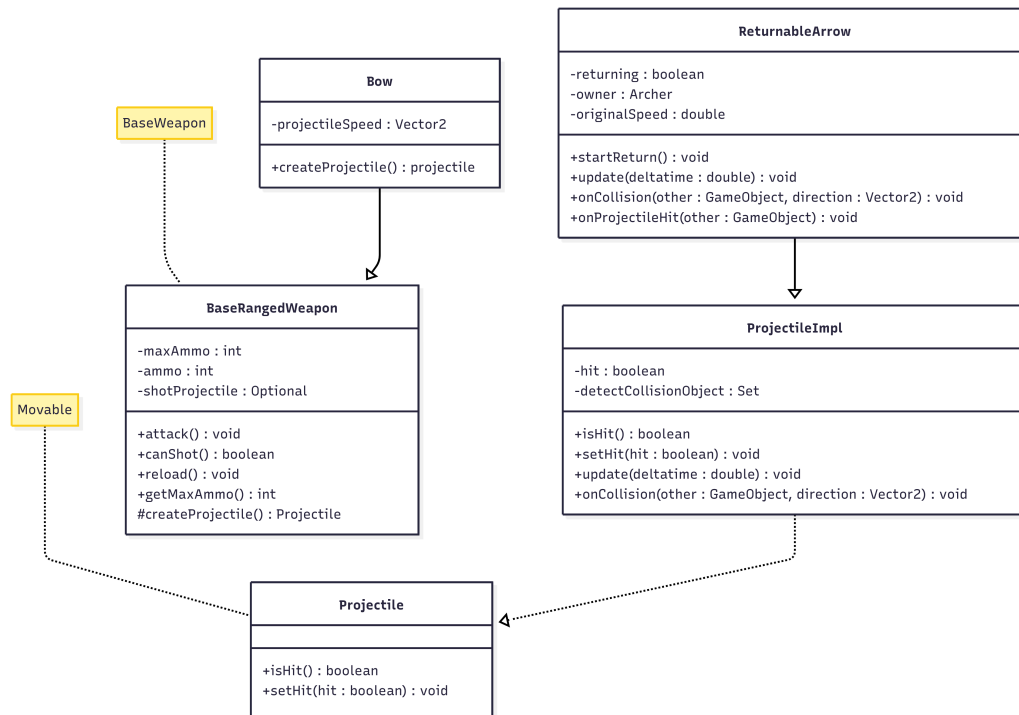


Figura 2.23: Schema UML rappresentante il sistema di armi a distanza e proiettili

Problema Nel gioco era necessario dotare i personaggi di armi a distanza, capaci di:

- sparare proiettili personalizzati (es. frecce);
- gestire munizioni limitate;
- rispettare tempi di ricarica (cooldown) tra un colpo e l'altro;
- creare meccanismi personalizzabili (es. frecce che ritornano all'arciere).

Inoltre, i proiettili stessi dovevano essere entità autonome e aggiornabili nel tempo, capaci di reagire alle collisioni.

Soluzione Il sistema è stato progettato in maniera modulare:

- **BaseRangedWeapon**: classe astratta che gestisce logica comune delle armi a distanza (cooldown, ammo, attack, onAttack), delegando a sottoclassi la creazione del proiettile (createProjectile).

- **Bow**: arma concreta usata dall'arciere, che produce oggetti **ReturnableArrow** alla pressione del comando di attacco.
- **ProjectileImpl**: implementazione generica di proiettile con logica di movimento, disegno, e reazione alle collisioni.
- **ReturnableArrow**: estensione di **ProjectileImpl** con comportamento speciale che consente alla freccia di tornare al mittente.

Questo design permette l'aggiunta di nuove armi e proiettili con comportamenti diversi, mantenendo separata la logica di **Weapon** e di **Projectile**.

Gestione delle abilità e comportamento avanzato del personaggio: **Archer**

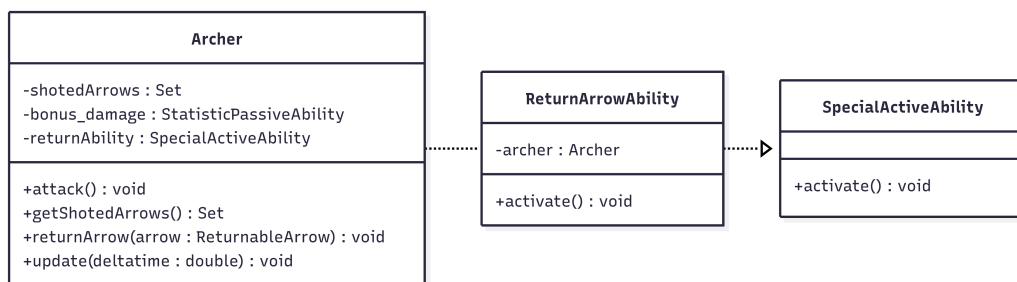


Figura 2.24: Schema UML rappresentante la classe Archer

Problema Era necessario rappresentare un personaggio giocabile capace di:

- utilizzare armi a distanza;
- interagire attivamente con i proiettili dopo averli lanciati (farli ritornare);
- beneficiare di effetti passivi basati sullo stato di gioco (es. numero di frecce attive);
- supportare un sistema flessibile di abilità attive e passive legate allo stile di gioco.

Soluzione È stata creata la classe **Archer**, estensione di **BaseCharacter**, che:

- equipaggia un **Bow** come arma;
- tiene traccia delle frecce lanciate;
- gestisce un'abilità attiva **ReturnArrowAbility**, che attiva il ritorno delle frecce (richiamando **startReturn()** su ogni **ReturnableArrow**);
- integra una abilità passiva **StatisticPassiveAbilityImpl**, che aumenta l'attacco proporzionalmente al numero di frecce in gioco.

Etichette grafiche per interfaccia utente

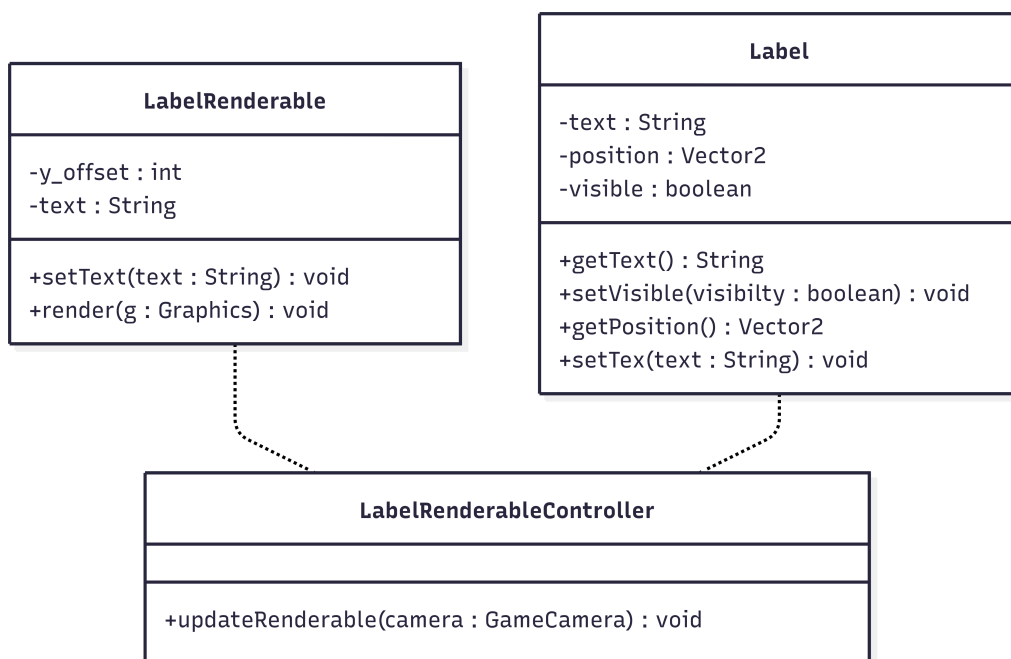


Figura 2.25: Schema UML rappresentante la classe **Label**

Problema Nel gioco era necessario un sistema per mostrare a schermo:

- messaggi testuali (es. statistiche, notifiche);
- etichette statiche o temporanee che non dipendono dal mondo di gioco, ma dalla GUI;

- un modo per tenere sincronizzato il contenuto dell'etichetta tra Model e View.

Il sistema doveva essere leggero, modulare e riutilizzabile, e compatibile con l'architettura MVC già adottata nel gioco.

Soluzione Sono stati progettati tre componenti distinti, secondo il *pattern Model-View-Controller*:

- **Label**: rappresenta una semplice etichetta con posizione, visibilità, e contenuto testuale.
- **LabelRenderable** (*View*): si occupa di disegnare il testo usando Java 2D, con un offset verticale.
- **LabelRenderableController**: sincronizza *View* e *Model*, aggiornando il testo e la visibilità ogni frame.

Suoni di gioco

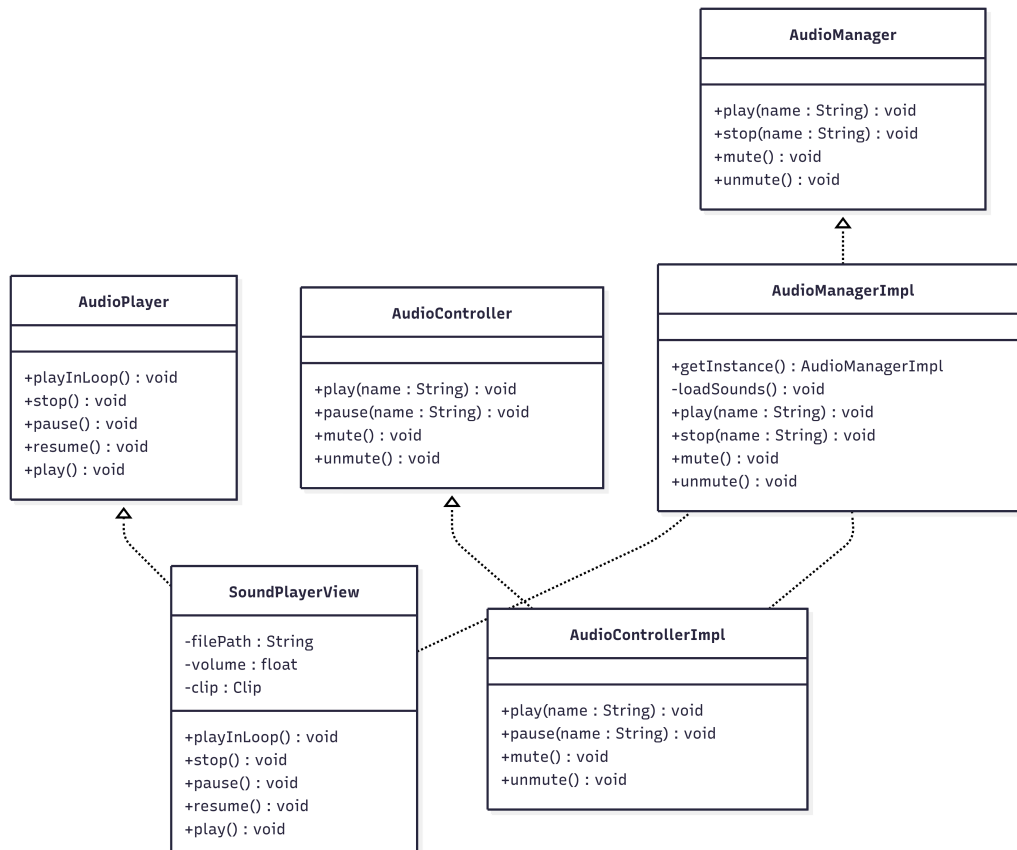


Figura 2.26: Schema UML rappresentante il sistema di audio

Problema Bisognava implementare un sistema per poter gestire la musica del gioco ma anche che potesse essere efficiente, e soggetto a poche modifiche, per l'implementazione futura di altri suoni quali:

- Suoni dei nemici.
- Suono per il movimento del personaggio.
- Suono delle armi.

Soluzione Si è deciso di adottare un sistema strutturato su più classi cercando di attenersi il più possibile al *pattern MVC*. La struttura è la seguente:

- Una classe `SoundPlayerView` che implementando `AudioPlayer` permette la manipolazione delle tracce audio(es. `play`, `pause`, `reset`). Garantendo inoltre la possibilità di riprodurre tali tracce per un numero di `Loop` a scelta.
- Una classe `AudioManagerImpl`, che implementa `AudioManger`, che può gestire tutte le tracce audio del gioco, permettendo di caricarle in una mappa assegnando a ciascuna traccia una chiave che la identifica. Dato che per ora il programma presenta solo la musica, ho scelto di adottare il pattern `Singleton`, in modo che non possano essere create più istanze del manager così che non ci sia il rischio di far partire involontariamente la stessa traccia più volte e sovrapporle.
- Una classe `AudioController` che permetta di controllare il manager così da rispettare il più possibile il pattern `MVC`.

Menu del gioco

Problema Serviva un menu che si aprisse all'apertura del gioco, con la possibilità di uscire da esso o di avviare la partita.

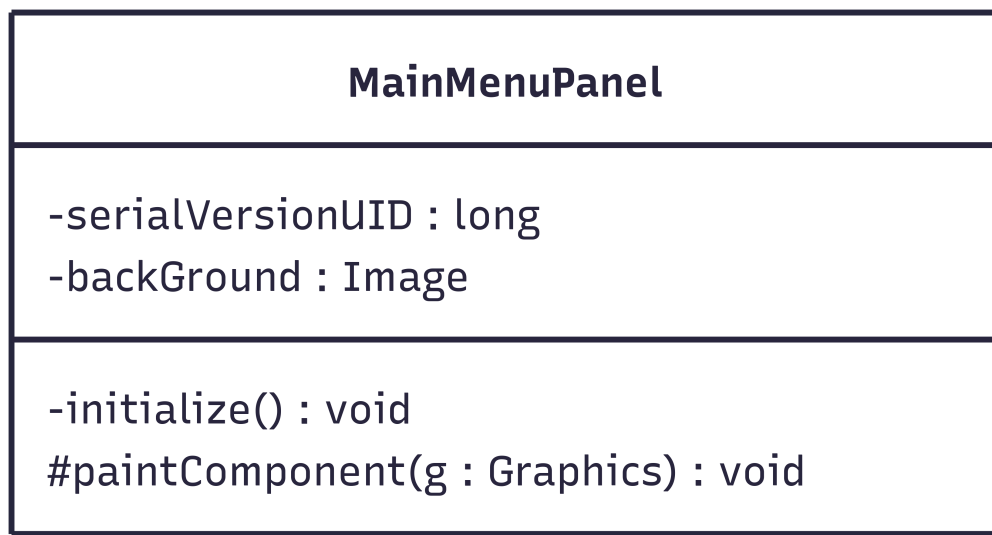


Figura 2.27: Schema UML rappresentante il menù di gioco

Soluzione È stata introdotta la classe `MainMenuPanel` che estende `JPanel`, utilizza un `GridBagLayout` per posizionare una `JLabel` col titolo del gioco e dei `JButton` per i bottoni `start` e `exit` al centro dello schermo. Dopo di

che usa `paintComponent` per disegnare uno sfondo per il menu. All'avvio del gioco vengono creati degli `ActionListener` per i bottoni che permettono di avviare la partita oppure uscire dall'applicazione.

Capitolo 3

Sviluppo

3.1 Test automatizzati

Per utilizzare Test senza interazione dell'utente per le classi del model, l'applicazione utilizza la suite *JUnit*. La maggior parte degli elementi del Controller e la View non sono stati testati per problemi di complessità. Riportiamo brevemente delle descrizioni i componenti sottoposti ai test:

- **ArcherTest**: testa il funzionamento dell'arciere;
- **TestBlocks**: testa i blocchi di lava e i blocchi di liane;
- **TestBuffManager**: testa la gestione dell'applicazione e della rimozione dei potenziamenti;
- **TestCaster**: testa il funzionamento del mago;
- **TestCollisions**: testa le collisioni fra oggetti;
- **TestCustomTimer**: testa il funzionamento dei timer;
- **TestDruid**: testa il funzionamento del druido;
- **TestEnemy**: testa il funzionamento dei nemici tranne il loro movimento;
- **GameCamera**: testa il corretto posizionamento della telecamera;
- **TestManagerAndFamiliar**: testa il corretto funzionamento del manager e famiglio, soprattutto controllando la comunicazione tra le classi;
- **TestMeleeWeapon**: testa il funzionamento delle armi da mischia;

- **TestMerchant**: testa il funzionamento del negozio e della sua entrata;
- **TestMovable**: testa il movimento degli oggetti;
- **TestPhysics**: testa la fisica del gioco, applicazione di gravità e risoluzioni di collisioni tra entità e blocchi;
- **TestProjectile**: testa il movimento e la capacità di colpire altri oggetti dei proiettili;
- **TestRogueAbility**: testa il funzionamento del ladro;
- **TestSprings**: testa il salvataggio del gioco, il cambio del personaggio e la ricarica completa di vita e mana;
- **TestTemporaryStatistics**: testa le statistiche temporanee;
- **TestTimerManager**: testa la gestione dei timer;

3.2 Note di sviluppo

3.2.1 Davide Mancini

Molte parti di codice sono state prese da vecchi progetti personali. Il game loop è stato costruito prendendo ispirazione da Game as Lab del Professor Ricci.

Utilizzo di Pair dalla libreria Apache Commons

Utilizzato solo in `AbstractCollisionsManager` per ottenere una chiave per identificare due game objects che stanno collidendo, utilizzata per la mappa delle collisioni nel frame precedente. Viene utilizzata per notificare i game objects con `onCollisionExit()` dopo aver concluso una collisione. Viene utilizzato in <https://github.com/FireSoul04/00P24-fall-to-hell/blob/5c96d0e56249127deaab86c80ed66cda7a4dce2b/src/main/java/it/unibo/falltohell/model/impl/manager/AbstractCollisionsManager.java#L26C1-L26C101>

Utilizzo di Optional e Lambda expressions

Utilizzato in molteplici punti. Un esempio è <https://github.com/FireSoul04/00P24-fall-to-hell/blob/5c96d0e56249127deaab86c80ed66cda7a4dce2b/src/main/java/it/unibo/falltohell/model/impl/gameobject/weapons/BaseWeapon.java#L71>

Utilizzo di Reflection

Utilizzato nel coltello lanciabile dal ladro. Serve per controllare quale game object può fermare e distruggere il coltello mentre è in lancio e cosa invece sarà ignorato e passato attraverso. Viene utilizzato in <https://github.com/FireSoul04/00P24-fall-to-hell/blob/5c96d0e56249127deaab86c80ed66cda7a4dce2b/src/main/java/it/unibo/falltohell/model/impl/gameobject/movable/projectile/Knife.java#L21>

3.2.2 Martina Malagoli

Utilizzo di Optional

Gli Optional sono stati utilizzati varie volte. Un esempio è: <https://github.com/FireSoul04/00P24-fall-to-hell/blob/5c96d0e56249127deaab86c80ed66cda7a4dce2b/src/main/java/it/unibo/falltohell/model/impl/gameobject/entrance/ShopEntrance.java#L19>

Utilizzo di Stream e Lambda expressions

Streams e lambda expressions sono stati utilizzati più volte. Un esempio è: <https://github.com/FireSoul04/00P24-fall-to-hell/blob/5c96d0e56249127deaab86c80ed66cda7a4dce2b/src/main/java/it/unibo/falltohell/model/impl/gameobject/MerchantImpl.java#L70>

Utilizzo di Reflection

Utilizzata nelle Fireball per determinare con quali oggetti ignorare la collisione in: <https://github.com/FireSoul04/00P24-fall-to-hell/blob/5c96d0e56249127deaab86c80ed66cda7a4dce2b/src/main/java/it/unibo/falltohell/model/impl/gameobject/movable/projectile/Fireball.java#L28>

3.2.3 Sara Visani

Utilizzo di Lambda expressions, Stream e Optional

Le lambda, gli stream e gli Optional sono usate spesso. Un esempio è in Base enemy: <https://github.com/FireSoul04/00P24-fall-to-hell/blob/00f50d2cf307756b051d06d7b87207078f0ddd94/src/main/java/it/unibo/falltohell/model/impl/gameobject/movable/entity/enemy/BaseEnemy.java#L360>

Utilizzo del Generico

Il generico è stato utilizzato in varie classi separate all'interno della gerarchia dei Builder delle statistiche: <https://github.com/FireSoul04/00P24-fall-to-hell/blob/00f50d2cf307756b051d06d7b87207078f0ddd94/src/main/java/it/unibo/falltohell/model/impl/builder/GroundEnemyStatBuilderImpl.java#L23>

3.2.4 Lorenzo Casadei

Utilizzo di Optional

Uso di Optional con method reference, usato per aggiungere dinamicamente un proiettile alla lista delle frecce in volo. Permalink: <https://github.com/FireSoul04/00P24-fall-to-hell/blob/ea2e77ac831004d138e6898449eec7c2f60e9299/src/main/java/it/unibo/falltohell/model/impl/gameobject/movable/entity/character/Archer.java#L77>

Utilizzo di Lambda expression

Uso di lambda expression, utilizzato per definire un abilità passiva il cui effetto dipende dallo stato runtime dell'arciere. Permalink: <https://github.com/FireSoul04/00P24-fall-to-hell/blob/ea2e77ac831004d138e6898449eec7c2f60e9299/src/main/java/it/unibo/falltohell/model/impl/gameobject/movable/entity/character/Archer.java#L61>

Utilizzo di Stream e Lambda expression

Uso di stream con lambda expression per poter determinare se l'oggetto presenta le collisioni. Permalink: <https://github.com/FireSoul04/00P24-fall-to-hell/blob/ea2e77ac831004d138e6898449eec7c2f60e9299/src/main/java/it/unibo/falltohell/model/impl/gameobject/movable/projectile/ProjectileImpl.java#L95>

Inoltre ci terrei a precisare che le classi InputListener e SoundPlayerView sono state prese da un vecchio progetto di un altro membro del gruppo, tali classi sono state opportunamente modificate per evitare errori e gli sono stati aggiunti i commenti per spiegarne il funzionamento.

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Davide Mancini

Personalmente, ritengo un progetto con cui inizialmente sono riuscito a progettare, successivamente il progetto si è ingrandito, rendendo delle scelte di design difficili da tornare indietro. Penso di aver trascurato una buona progettazione iniziale ed è finita in una malprogettazione delle classi verso la conclusione dello sviluppo. Una cosa che considero un fallimento nella mia parte è stato l'utilizzo dell'algoritmo AABB (Axis-Aligned Bounding Box) per i controlli della collisione, poiché è molto semplice, ma che mi ha reso più difficile implementare la risoluzione delle collisioni tra entità e oggetti solidi, specialmente gestire la gravità. Poteva essere migliorata utilizzando una variante del suo algoritmo chiamata Swept AABB, che permette di bloccare completamente i movimenti successivi a una collisione e spinge un'entità nella direzione opposta alla collisione, anche se eventualmente è possibile implementare un collisions manager con quell'algoritmo in futuro. Ho imparato con questo progetto meglio il funzionamento di MVC e i suoi punti forza, anche se non sono sicuro che sia stato architettato bene e flessibile. Un'altra cosa che poteva essere sviluppata meglio è il `LevelBuilder`, che richiede un ordine preciso per funzionare. Ho notato come i test possono salvare tempo durante lo sviluppo. Nonostante non sia molto soddisfatto di come sia uscito il design del codice, sono soddisfatto del risultato e sarei volenteroso a portare avanti il progetto.

4.1.2 Martina Malagoli

All'inizio del progetto avevo grandi aspettative sulla sua riuscita, pensando che avremmo potuto e inserire tantissime diverse funzionalità senza alcun problema. Nel corso del lavoro mi sono però resa conto delle difficoltà e delle complessità che possono sorgere anche nel fare cose che possono sembrare inizialmente semplici e come, spesso, sia meglio partire a piccoli passi e poi ampliare le vedute. Inoltre ho riscontrato diverse difficoltà nel riuscire ad avere una visione d'insieme e di capire tutti i collegamenti fra le varie classi. Ho visto però che per superare questi problemi lavorare in gruppo è molto utile, in quanto comunicando si riescono a chiarire molti dubbi e insicurezze rispetto al lavoro che si sta svolgendo insieme. Allo stesso tempo, per quanto ci siano state difficoltà, fare questo progetto è stato molto stimolante e mi ha dimostrato come sia in grado di gestire un lavoro di gruppo e di portare a termine i compiti che mi vengono assegnati. Quindi in generale, per quanto penso che avrei potuto fare meglio alcune parti (cosa di cui mi sono resa conto in corso d'opera) mi ritengo abbastanza soddisfatta del lavoro svolto.

4.1.3 Sara Visani

Nel corso di questo progetto, mi sono impegnato al massimo, affrontando le diverse sfide con dedizione, nonostante la difficoltà. È stata un'esperienza di apprendimento intensa che mi ha permesso di esplorare e implementare diverse logiche di programmazione. Un'area in cui ritengo di aver incontrato le maggiori difficoltà è stata la gestione e l'implementazione delle classi relative alle Abilità. Credo che la loro struttura e il loro comportamento avrebbero potuto essere gestiti in modo più efficiente e pulito. Per quanto riguarda il movimento autonomo dei mostri, essendo stata la mia prima volta nell'affrontare un problema del genere, il risultato è rimasto leggermente "buggato". Nonostante ciò, sono soddisfatto del fatto che la logica di base funzioni e rappresenti un solido punto di partenza per futuri miglioramenti. Ritengo di aver svolto un buon lavoro, invece, nell'implementazione dei proiettili, del sistema di drop dai mostri e dei vari pattern Factory, in particolare quelli per la creazione di statistiche e mostri. La loro architettura mi sembra solida e ben strutturata. Sono particolarmente soddisfatto dell'implementazione del mio personaggio e, soprattutto, del sistema del famiglia. Ho investito molto tempo e attenzione in questi aspetti e credo che il risultato finale sia riuscito a cogliere appieno le mie intenzioni di design. In generale, nonostante le sfide e le aree in cui avrei potuto fare di meglio, mi sento soddisfatto per il lavoro che ho svolto. Questo progetto mi ha permesso di crescere, sia come programmatore che nella gestione di un compito complesso.

4.1.4 Lorenzo Casadei

Per quanto riguarda l'autovalutazione del mio lavoro sono soddisfatto fino a un certo punto, avevo già provato a programmare dei piccoli giochi da solo ma mai usando il pattern MVC che per me è stata la sfida più grande. Ho avuto difficoltà a capire come disporre le classi e come farle “comunicare” le une con le altre. Inoltre, nonostante le classi funzionassero, mi sono accorto verso la fine di non avere usato nessun tipo di pattern. Un problema che avrei potuto evitare se mi fossi concentrato di più sullo stile delle classi piuttosto che il loro semplice funzionamento. Come ultima cosa esattamente come quando feci l'esame di OOP faccio ancora molta fatica ad utilizzare le funzionalità avanzate del linguaggio, soprattutto gli stream. La mia intenzione sarebbe quella di continuare il progetto. Mi piacerebbe aggiungere più suoni sfruttando l'audio manager, vorrei implementare le animazioni (cosa che verso la fine stavo provando a fare ma che ho interrotto a causa della mancanza di sprite). Vorrei espandere il livello e magari aggiungerne altri, facendo anche un boss che possa avere un comportamento complesso, con diversi attacchi.

4.2 Difficoltà incontrate e commenti per i docenti

4.2.1 Davide Mancini

Una maggior difficoltà è stata recuperare il lavoro risultato dall'abbandono di un compagno di gruppo. Principalmente, mi aspettavo di affrontare meno difficoltà, però ho notato con il passare del tempo di aver dedicato molto più tempo al debug del previsto nonostante una piccola esperienza nel campo dei videogiochi. Una sfida è stata anche attendere la conclusione delle parti degli altri compagni di progetto e adattarmi al codice scritto da altre persone.

4.2.2 Martina Malagoli

Una delle difficoltà più grandi è stata essere in grado di gestire il tempo e di comprendere i vari collegamenti fra le diverse parti del lavoro. Inoltre l'assenza di un compagno di gruppo ha reso sicuramente più complessa tutta la realizzazione del progetto.

Appendice A

Guida utente

A.1 Obiettivo del Gioco

L'obiettivo del gioco è scendere in profondità per salvare Persefone, attraversando i vari pericoli e sopravvivere. In questa demo è di semplicemente uccidere i nemici senza morire.

Glossario dei Tasti

- W: muovere verso l'alto il famigliaio
- A: muovere verso sinistra il personaggio e il famigliaio
- S: muovere verso basso il famigliaio
- D: muovere verso destra il personaggio e il famigliaio
- SHIFT: utilizzare l'abilità speciale per tutte le classi tranne quella del druido
- SPACE: per saltare
- Q: utilizzare la speciale del druido e del mago
- E: per usare l'attacco base
- F: per interagire con gli oggetti
- P: per mettere in pausa
- O: per togliere la pausa

- C + direzione: serve per attivare l'attacco del famiglia insieme ad una direzione (inoltre se è verso il basso vale anche dove il personaggio sta guardando)

Glossario Entità

- Giocabili
 - Rogue (Ladro): è un personaggio che sfrutta una daga per attaccare i nemici a distanza ravvicinata, presenta una abilità attiva che gli permette di lanciare tre coltelli in tre direzioni differenti (orizzontale e obliqua verso alto e basso). Presenta anche due abilità passive, una è il doppio salto che gli permette di raggiungere altezze più elevate. Mentre la seconda gli permette di ottenere più velocità di movimento nel momento in cui viene colpito da un nemico.
 - Archer (Arciere): è l'unico personaggio del gioco che non utilizza mana, in quanto la sua abilità è parte integrante del suo stile di gioco. Il personaggio utilizza un arco per sparare delle frecce, di cui ha un numero massimo, ognuna delle quali poi può essere richiamata attraverso l'abilità. Le frecce richiamate seguono il giocatore fino a tornare da lui e ricaricare le sue munizioni. Ogni freccia fa danno anche nel momento in cui sta tornando e per incentivare l'utilizzo dell'abilità è stata aggiunta un'abilità passiva che aumenta il danno sulla base di quante frecce sono già state lanciate. Quindi nel momento in cui le frecce ritornano dal personaggio fanno più danno ai nemici.
 - Caster (Mago): è il personaggio che basa tutto il suo gioco sull'utilizzo del mana. Egli infatti è in grado di lanciare palle di fuoco e fare un potente attacco esplosivo che danneggia i nemici vicini. Inoltre può curare sé stesso. L'utilizzo costante di mana è facilitato dalla sua capacità di recuperarlo col passare del tempo. Nel caso in cui questo però non sia più sufficiente per utilizzare gli attacchi magici, il caster può attaccare con la sua staffa i nemici.
 - Druid (Druido): questo personaggio è un'attaccante ravvicinato. Come abilità passiva ha una rigenerazione ogni volta che uccide dei nemici, più ne uccide più la percentuale si alza, ma se non uccide per un periodo le uccisioni si resettano. La speciale invece evoca un famiglia che vola vicino ad esso e quando comandato attacca nella direzione indicata, se ne possono evocare finché si ha

mana e attaccare con loro finché si ha mana. Verso il basso conta anche dove il giocatore sta guardando perché prima si muove di lato poi in basso.

- Interagibili

- Drop (Bottino): quando il nemico muore c'è una possibilità che lascia un bottino che, se recuperato in tempo, applica un buff (potenziamento) a una caratteristica del personaggio.
- Potion (Pozione): oggetto acquistabile nel negozio con i punti del giocatore in grado di dare potenziamenti temporanei al personaggio corrente.
- Save Point (Lapide): oggetto che permette di salvare la partita quando ci interagisci.
- Character Changer (Cristallo): oggetto che permette di cambiare il personaggio interagendoci.

- Nemici

- Imp (Folletto Maligno): nemico di base di terra come i vari nemici insegue il giocatore. (Colore giallo)
- Centaur (Centauro): nemico avanzato di terra come i vari nemici insegue il giocatore. (Colore blu)
- Lotawiec: nemico avanzato volante, spara proiettili che inseguono il giocatore, come i vari nemici insegue il giocatore. (Colore rosso)
- Tengu: nemico di base volante, spara proiettili verso il basso senza nessuna mira, come i vari nemici insegue il giocatore. (Colore viola)

- Blocchi e Aree Speciali

- Lava Block (Blocco di lava): blocco che toglie vita (senza considerare eventuale vita temporanea) al personaggio e ai nemici ogni volta che viene toccato.
- Vine Block (Blocco di liane): blocco che riduce la velocità di movimento del personaggio e dei nemici una volta toccato.
- Zone Sicure: stanze in cui il giocatore è al sicuro. Queste sono il negozio dove il mercante vende le pozioni e le terme dove il giocatore recupera vita e mana, e dove può cambiare personaggio e salvare.

- NPC (Personaggi non Giocabili)
 - Merchant (Mercante): è un personaggio non giocabile che è presente nel mercato con cui non si può interagire.
- Punteggio
 - Punti Anima: sono i punti guadagnati uccidendo i nemici. Questi punti servono per comprare le pozioni dal mercante.

Appendice B

Esercitazioni di laboratorio

B.1 `davide.mancini10@studio.unibo.it`

- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=177162#p246021>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=178723#p247194>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=179154#p248089>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=180101#p249551>
- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=181206#p250874>

B.2 `martina.malagoli4@studio.unibo.it`

- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=177162#p246175>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=178723#p247282>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=179154#p248356>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=180101#p249552>

- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=181206#p250909>